Memory Management

Chapter 3

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory

Memory manager

The part of the operating system that manages the memory hierarchy.

- Keep track of which parts of memory are in use
- Allocate memory to processes when they need it
- Deallocate memory when the processes are done

Memory

Without a Memory Abstraction

With a Memory Abstraction

- Base and Limit Registers
- Swapping
- Virtual Memory

No Memory Abstraction

Early computers had no memory abstraction. Every program simply saw the physical memory.

MOV AX, 20

The value stored at physical memory address 20 is directly loaded into the AX register (direct access to the main memory).

```
-d 20
11B4:0020  FF FF FF FF FF FF FF FF-FF FF FF FF 00 0C 4E 01   ..............N.
11B4:0030  D8 10 14 00 18 00 B4 11-FF FF FF FF 00 00 00 00   ................
11B4:0040  06 16 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
11B4:0050  CD 21 CB 00 00 00 00 00-00 00 00 00 00 20 20 20   .!............
11B4:0060  20 20 20 20 20 20 20 20-00 00 00 00 00 20 20 20           .....
11B4:0070  20 20 20 20 20 20 20 20-00 00 00 00 00 00 00 00           ........
11B4:0080  00 0D 54 45 4D 50 3D 43-3A 5C 44 4F 53 0D 3A 5C   ..TEMP=C:\DOS.:\
11B4:0090  44 4F 53 0D 00 00 00 00-00 00 00 00 00 00 00 00   DOS.............
-a 100
11B4:0100 MOV AX, [20]
11B4:0103
-r
AX=0000  BX=0000  CX=0000  DX=0000  SP=FFEE  BP=0000  SI=0000  DI=0000
DS=11B4  ES=11B4  SS=11B4  CS=11B4  IP=0100   NV UP EI PL NZ NA PO NC
11B4:0100 A12000        MOV     AX,[0020]                        DS:0020=FFFF
-t

AX=FFFF  BX=0000  CX=0000  DX=0000  SP=FFEE  BP=0000  SI=0000  DI=0000
DS=11B4  ES=11B4  SS=11B4  CS=11B4  IP=0103   NV UP EI PL NZ NA PO NC
11B4:0103 8AFF          MOV     BH,BH
```

No Memory Abstraction

Intel 8088 (Real address mode):

MOV AX, [20]

DS: 11B4

OFFSET: 20

Memory Address: 11B40 + 20 = 11B60

```
-d 20
11B4:0020  FF FF FF FF FF FF FF FF-FF FF FF FF 00 0C 4E 01   ...............N.
11B4:0030  D8 10 14 00 18 00 B4 11-FF FF FF FF 00 00 00 00   ................
11B4:0040  06 16 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
11B4:0050  CD 21 CB 00 00 00 00 00-00 00 00 00 00 20 20 20   .!............
11B4:0060  20 20 20 20 20 20 20 20-00 00 00 00 00 20 20 20           .....
11B4:0070  20 20 20 20 20 20 20 20-00 00 00 00 00 00 00 00           ........
11B4:0080  00 0D 54 45 4D 50 3D 43-3A 5C 44 4F 53 0D 3A 5C   ..TEMP=C:\DOS.:\
11B4:0090  44 4F 53 0D 00 00 00 00-00 00 00 00 00 00 00 00   DOS.............
-a 100
11B4:0100 MOV AX, [20]
11B4:0103
-r
AX=0000  BX=0000  CX=0000  DX=0000  SP=FFEE  BP=0000  SI=0000  DI=0000
DS=11B4  ES=11B4  SS=11B4  CS=11B4  IP=0100   NV UP EI PL NZ NA PO NC
11B4:0100 A12000        MOV     AX,[0020]                       DS:0020=FFFF
-t

AX=FFFF  BX=0000  CX=0000  DX=0000  SP=FFEE  BP=0000  SI=0000  DI=0000
DS=11B4  ES=11B4  SS=11B4  CS=11B4  IP=0103   NV UP EI PL NZ NA PO NC
11B4:0103 8AFF          MOV     BH,BH
```

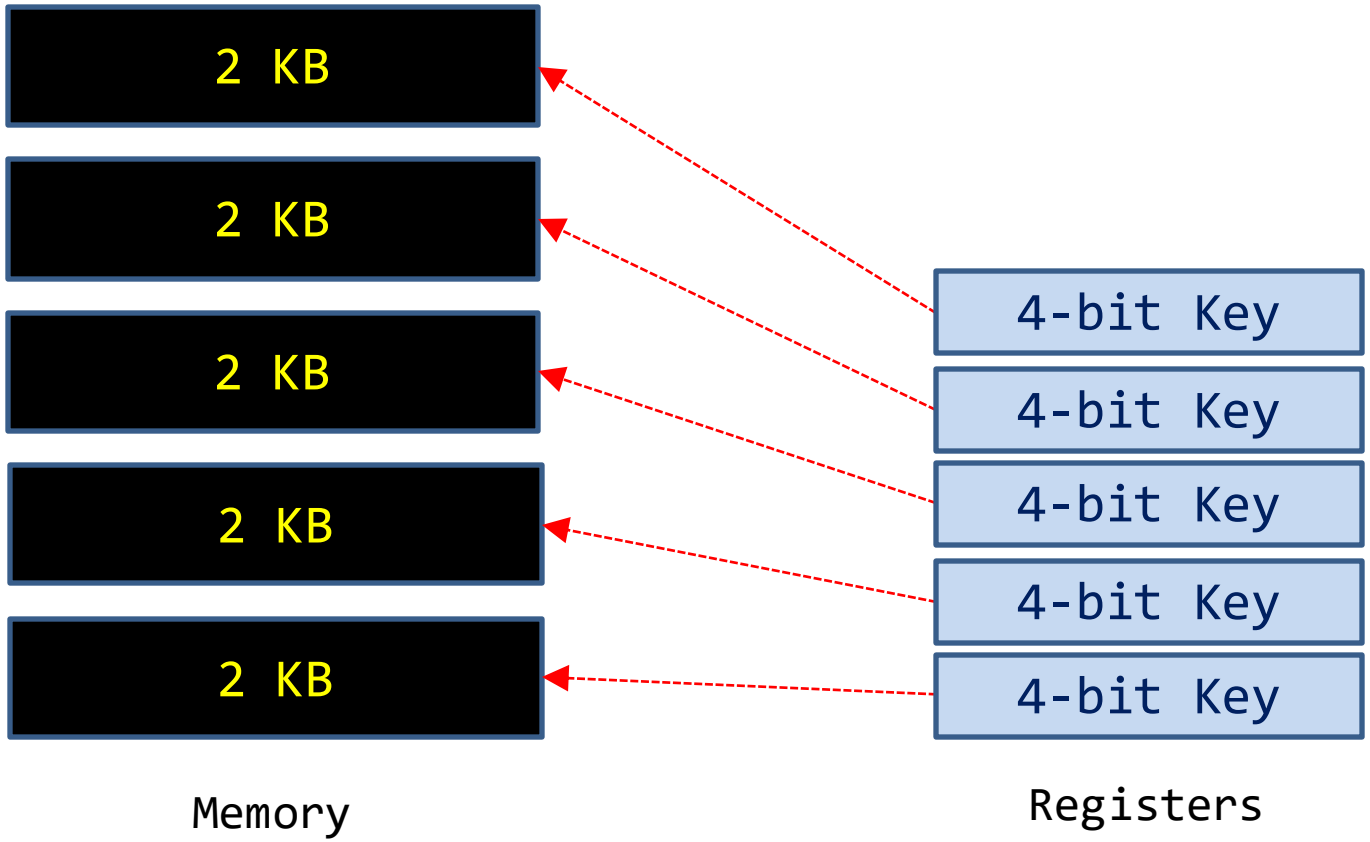
No Memory Abstraction

It is possible to run multiple programs at the same time,

e.g. the early models of the IBM 360 solved the problem as follows:

Memory was divided into 2-KB blocks and each was assigned a 4-bit protection key held in special registers inside the CPU.

No Memory Abstraction

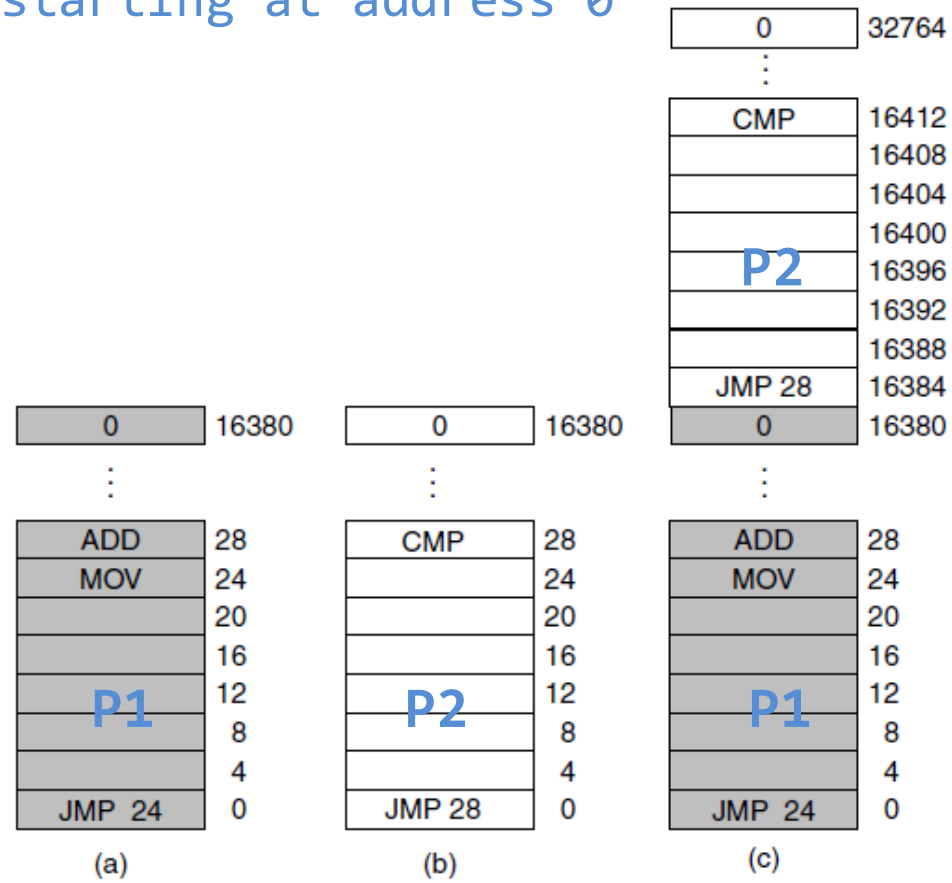


With 4-bit keys, only ? programs can be in memory at once

No Memory Abstraction

With no memory abstraction, is it possible to run multiple programs at the same time? Yes

Example: Two programs(P1, P2) each 16-KB and are loaded consecutively in memory starting at address 0

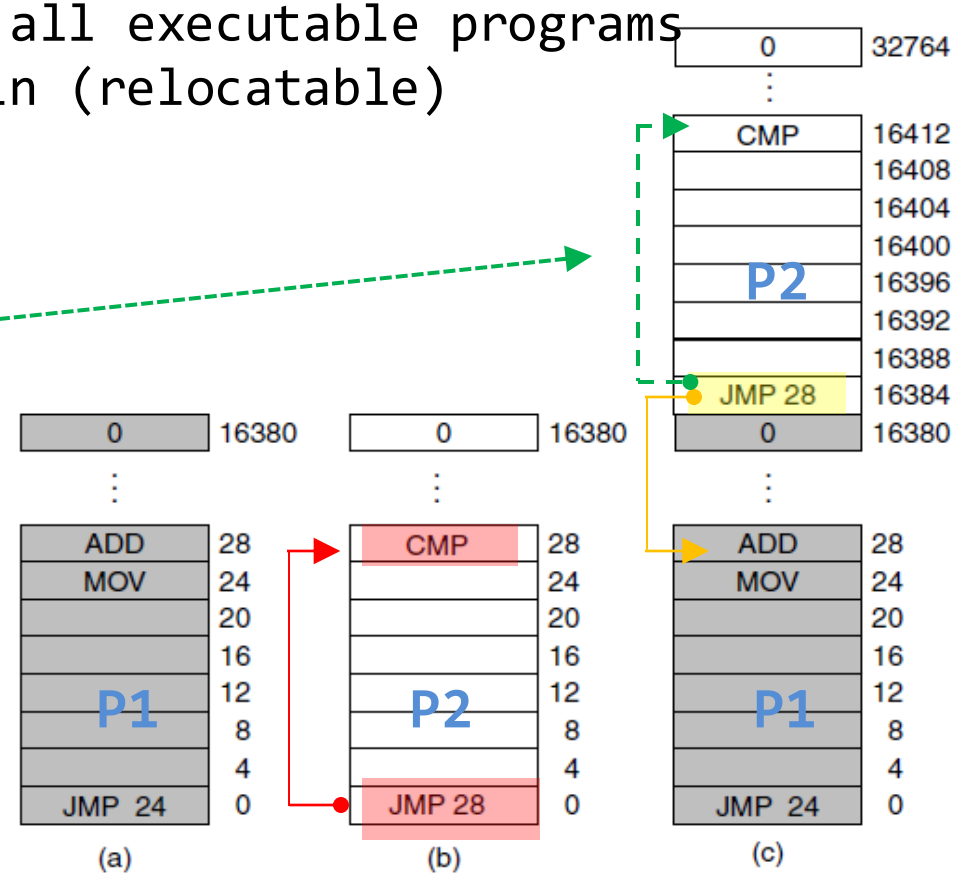


Example: Two programs(P1, P2) each 16-KB and are loaded consecutively in memory starting at address 0.

Relocation problem
Just consider first instruction in P2 (**JMP 28**) which jumps to the **ADD** instruction in the first program.

Relocation problem (solution): Not a general solution, requires extra information in all executable programs to indicate which words contain (relocatable) addresses and which do not.

JMP 28 + 16384 (base address)
JMP 16412 ⇒ CMP



A MEMORY ABSTRACTION

Intel 80286

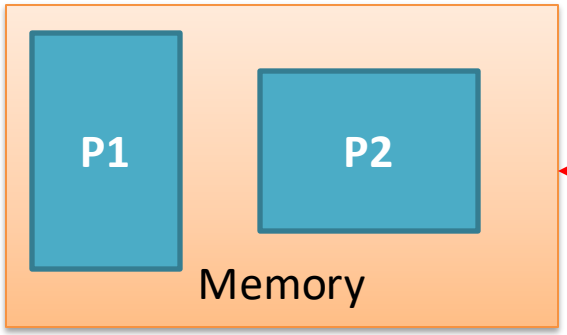
Real address mode
No memory protection

Protected virtual address mode
With memory protection

A MEMORY ABSTRACTION
ADDRESS SPACES

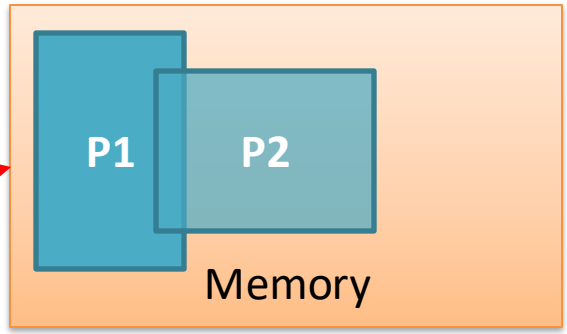
Two problems have to be solved to allow multiple applications to be in memory at the same time without interfering with each other: **protection and relocation**

An **address space** is the set of addresses that a process can use to address memory.



Each process has its own address space, independent of those belonging to other processes.

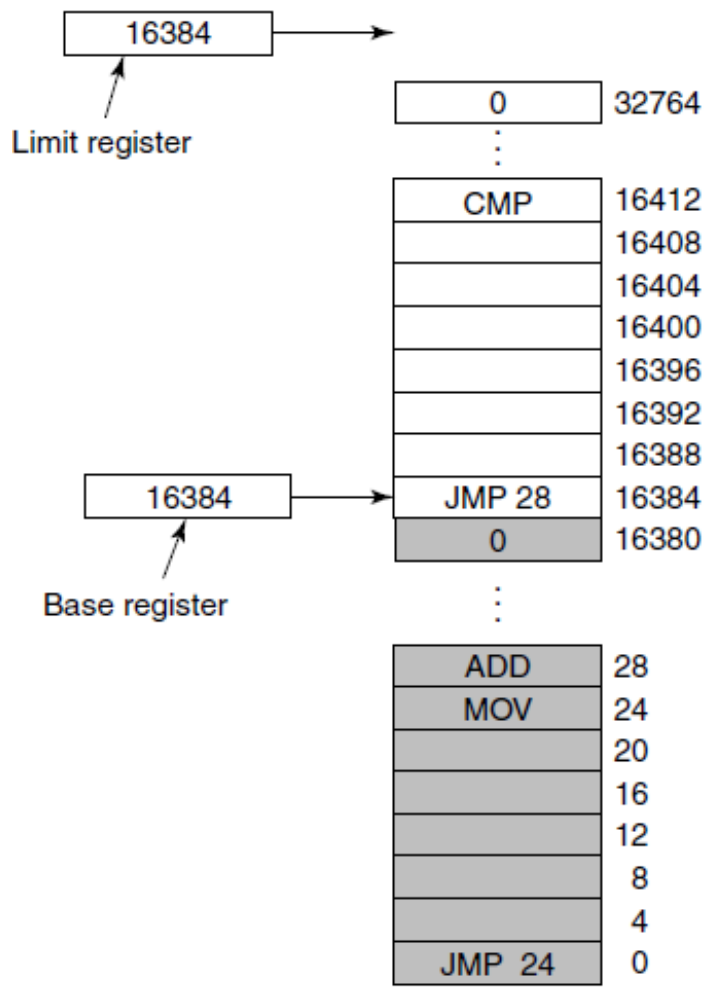
In some special circumstances where processes want to share their address spaces.



Base and Limit Registers

Base and limit registers:

Give each process a separate address space.



Base register

Loaded with the physical address where its program begins in memory.

Limit register

Loaded with the length of the program.

Base and Limit Registers

Base and limit registers were used in early memory management schemes to protect memory and provide a form of segmentation.

They are part of a simple method for ensuring that each process accesses only its own memory space.

When a process attempts to access memory, the address is first checked against the limit register to ensure it is within the allowed range. If it is, the base register is added to the address to obtain the actual physical address.

Base and Limit Registers

Relocation issue ?

Every time a process references memory (read/write), the CPU automatically adds the base value to the address generated by the process before sending the address out on the memory bus.

e.g.

JMP 28 ⇒ JMP 28 + base register

JMP 28 ⇒ JMP 16412

Disadvantage:

Needs to perform an addition and a comparison on every memory reference.

Memory overloading

On a typical Windows, OS X, or Linux system, something like 50–100 processes or more may be started up as soon as the computer is booted.

Windows 10

Task Manager

File Options View

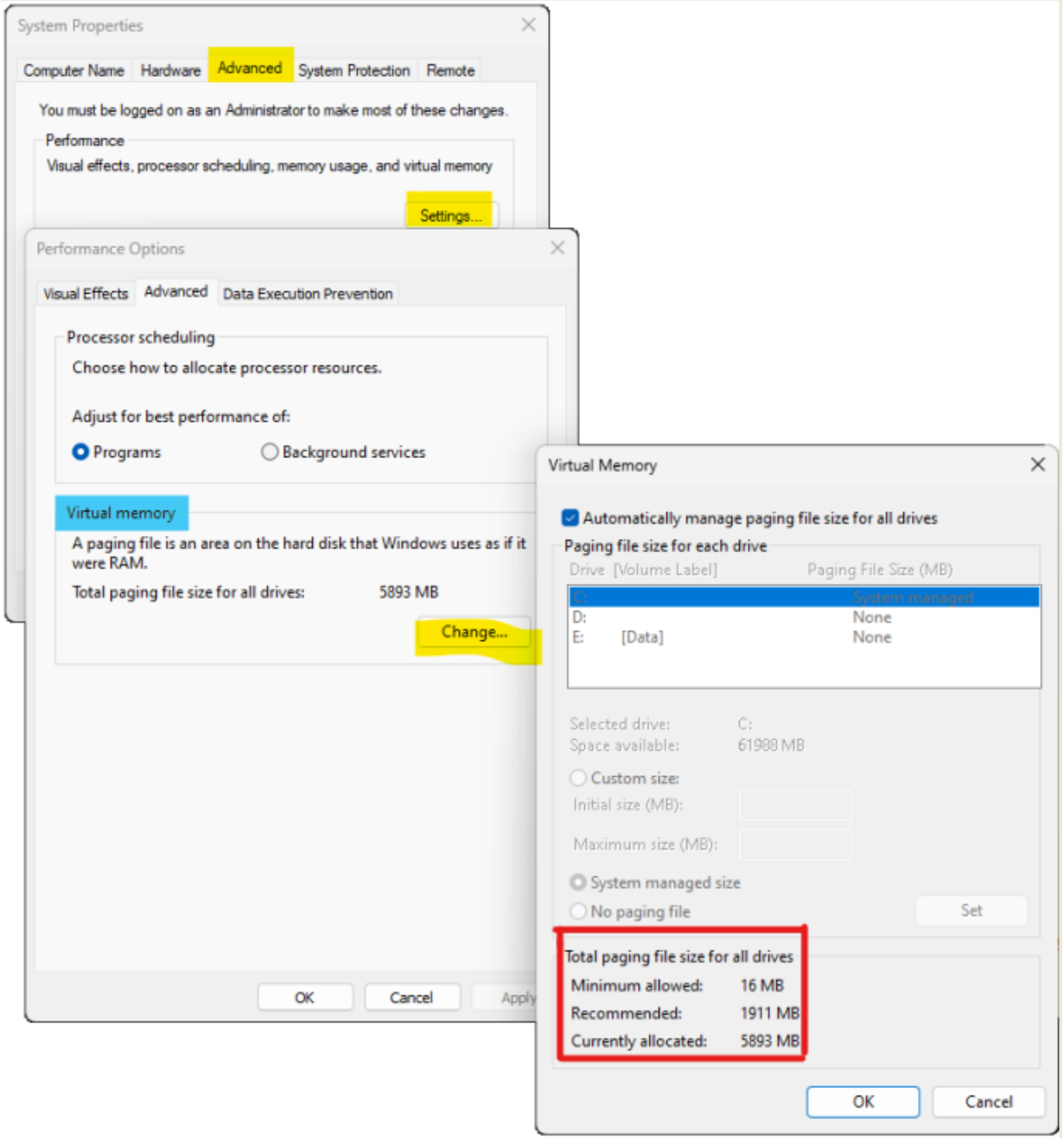
Processes | Performance | App history | Startup | Users | Details | Services

Name	PID	Description	Status	Group
WpnUserService_1c2879	6508	Windows Push Notifications User Service_1c2879	Running	UnistackSvcGroup
UserDataSvc_1c2879	5224	User Data Access_1c2879	Running	UnistackSvcGroup
UnistoreSvc_1c2879	5224	User Data Storage_1c2879	Running	UnistackSvcGroup
PimIndexMaintenanceSvc_...	5224	Contact Data_1c2879	Running	UnistackSvcGroup
OneSyncSvc_1c2879	5224	Sync Host_1c2879	Running	UnistackSvcGroup
CDPUserSvc_1c2879	6416	Connected Devices Platform User Service_1c2879	Running	UnistackSvcGroup
WSearch	9120	Windows Search	Running	
wscsvc	1560	Security Center	Running	LocalServiceNetworkRe
WpnService	3896	Windows Push Notifications System Service	Running	netsvcs
wlidsvc	1792	Microsoft Account Sign-in Assistant	Running	netsvcs
WlanSvc	2972	WLAN AutoConfig	Running	LocalSystemNetworkRe
Winmgmt	2416	Windows Management Instrumentation	Running	netsvcs
WinHttpAutoProxySvc	2308	WinHTTP Web Proxy Auto-Discovery Service	Running	LocalServiceNetworkRe
WinDefend	3884	Windows Defender Antivirus Service	Running	
WdNisSvc	5164	Windows Defender Antivirus Network Inspection Service	Running	
WdiSystemHost	9268	Diagnostic System Host	Running	LocalSystemNetworkRe
WdiServiceHost	4724	Diagnostic Service Host	Running	LocalService
wcncsvc	7668	Windows Connect Now - Config Registrar	Running	LocalServiceAndNoImp
Wcmsvc	2788	Windows Connection Manager	Running	LocalServiceNetworkRe
WbioSrvc	16936	Windows Biometric Service	Running	WbioSvcGroup
W32Time	1208	Windows Time	Running	LocalService
VaultSvc	768	Credential Manager	Running	
UsoSvc	6160	Update Orchestrator Service	Running	netsvcs
UserManager	2120	User Manager	Running	netsvcs
TrkWks	3872	Distributed Link Tracking Client	Running	LocalSystemNetworkRe
TPHKLOAD	3848	Lenovo Hotkey Client Loader	Running	
TokenBroker	7796	Web Account Manager	Running	netsvcs
TimeBrokerSvc	1324	Time Broker	Running	LocalServiceNetworkRe
Themes	1936	Themes	Running	netsvcs
TapiSrv	3268	Telephony	Running	NetworkService
TabletInputService	7128	Touch Keyboard and Handwriting Panel Service	Running	LocalSystemNetworkRe
SystemEventsBroker	932	System Events Broker	Running	DcomLaunch
SysMain	1952	Superfetch	Running	LocalSystemNetworkRe
SynTPEnhService	3856	SynTPEnh Caller Service	Running	

Fewer details | Open Services

Ubuntu 18.04.4 LTS

```
top - 10:28:51 up  2:48,  1 user,  load average: 0.81, 0.33, 0.23
Tasks: 300 total,   1 running, 254 sleeping,   0 stopped,   0 zombie
%Cpu(s):  4.4 us,  2.9 sy,  0.0 ni, 92.1 id,  0.2 wa,  0.0 hi,  0.4 si,  0.0 st
KiB Mem :  5557368 total,   773036 free,  2666120 used,  2118212 buff/cache
KiB Swap:  2097148 total,  2097148 free,        0 used.  2461024 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
 1994 rahim     20   0 1050800  85476  61792 S   9.2  1.5   7:47.87 Xorg
 2531 rahim     20   0  747368  38896  29360 S   3.0  0.7   0:01.45 gnome-flashback
 2516 rahim     20   0  854664  36668  28204 S   2.6  0.7   1:06.01 metacity
 5481 rahim     20   0 2055276 218952  79244 S   2.6  3.9   2:33.61 Telegram
 6762 rahim     20   0   51320   4116   3416 R   1.6  0.1   0:00.42 top
 2764 rahim     20   0 1198112 265184 133240 S   1.3  4.8   3:18.19 chrome
 1018 root      20   0  269700   5984   5240 S   1.0  0.1   1:53.22 iio-sensor-prox
 4723 rahim     20   0 29.128g 264456 111368 S   1.0  4.8  19:23.92 chrome
 6098 rahim     20   0  732788  40024  29548 S   0.7  0.7   0:03.23 gnome-terminal-
    1 root      20   0  226060   9780   6724 S   0.3  0.2   0:09.32 systemd
    8 root      20   0       0      0      0 I   0.3  0.0   0:38.70 rcu_sched
   16 root      20   0       0      0      0 S   0.3  0.0   0:04.48 ksoftirqd/1
  134 root      20   0       0      0      0 I   0.3  0.0   0:31.15 kworker/1:2
  375 root      20   0       0      0      0 I   0.3  0.0   0:20.38 kworker/2:2
 3764 rahim     20   0  562772  84812  64348 S   0.3  1.5   2:01.89 chrome
 4387 rahim     20   0 4826408 112556  76008 S   0.3  2.0   0:16.16 chrome
 4464 rahim     20   0 4933148 192940  98008 S   0.3  3.5   0:41.90 chrome
 4612 rahim     20   0 4873976 179960  92964 S   0.3  3.2   0:24.22 chrome
 4691 rahim     20   0 4921964 206344  98820 S   0.3  3.7   0:43.56 chrome
 5079 root      20   0       0      0      0 I   0.3  0.0   1:07.89 kworker/0:1
 6566 root      20   0       0      0      0 I   0.3  0.0   0:02.47 kworker/3:2
 6722 root      20   0       0      0      0 I   0.3  0.0   0:00.18 kworker/u8:0
    2 root      20   0       0      0      0 S   0.0  0.0   0:00.01 kthreadd
    6 root       0 -20       0      0      0 I   0.0  0.0   0:00.00 mm_percpu_wq
    7 root      20   0       0      0      0 S   0.0  0.0   0:00.31 ksoftirqd/0
    9 root      20   0       0      0      0 I   0.0  0.0   0:00.00 rcu_bh
   10 root      rt   0       0      0      0 S   0.0  0.0   0:00.00 migration/0
   11 root      rt   0       0      0      0 S   0.0  0.0   0:00.05 watchdog/0
   12 root      20   0       0      0      0 S   0.0  0.0   0:00.00 cpuhp/0
```



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory overloading

Two general approaches to deal with memory overloading are:

Swapping

- Consists of bringing in each process in its entirety, running it for a while, then putting it back on the disk.
- Idle processes are mostly stored on disk, so they do not take up any memory when they are not running.

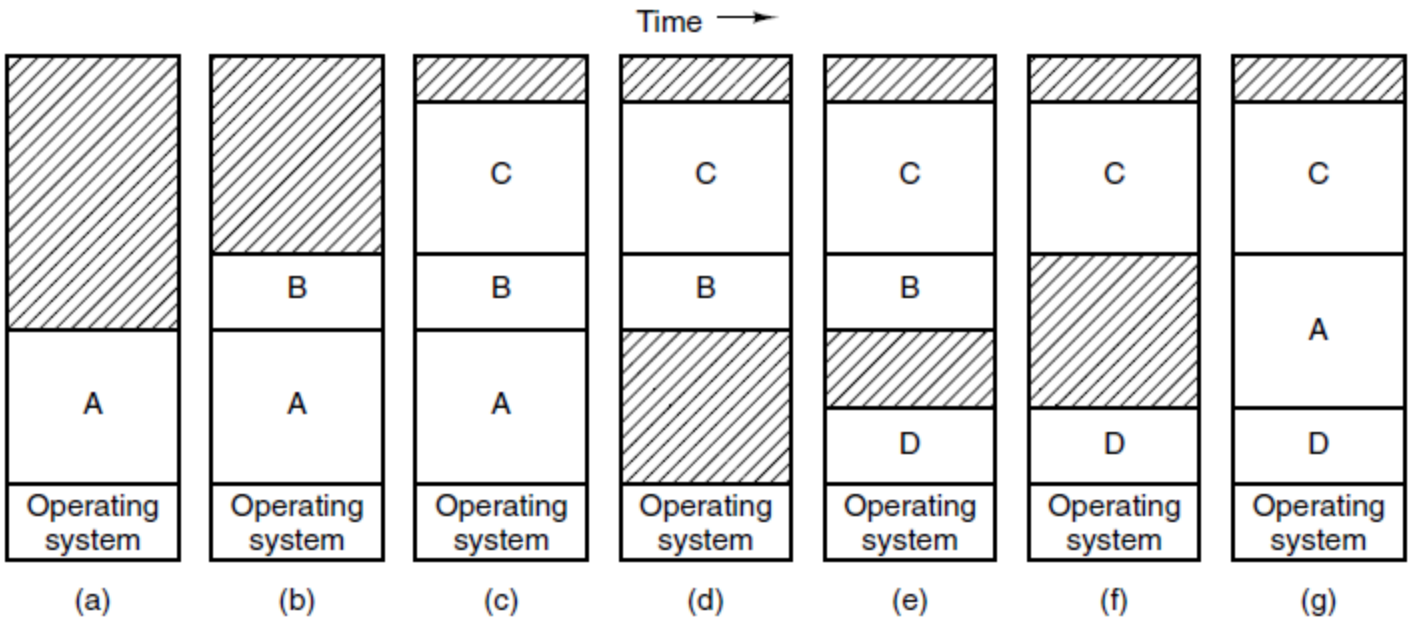
Virtual memory

Allows programs to run even when they are only partially in main memory.

Swapping

Consists of bringing in each process in its entirety, running it for a while, then putting it back on the disk.

<u>Swapping creates multiple holes in memory</u>.

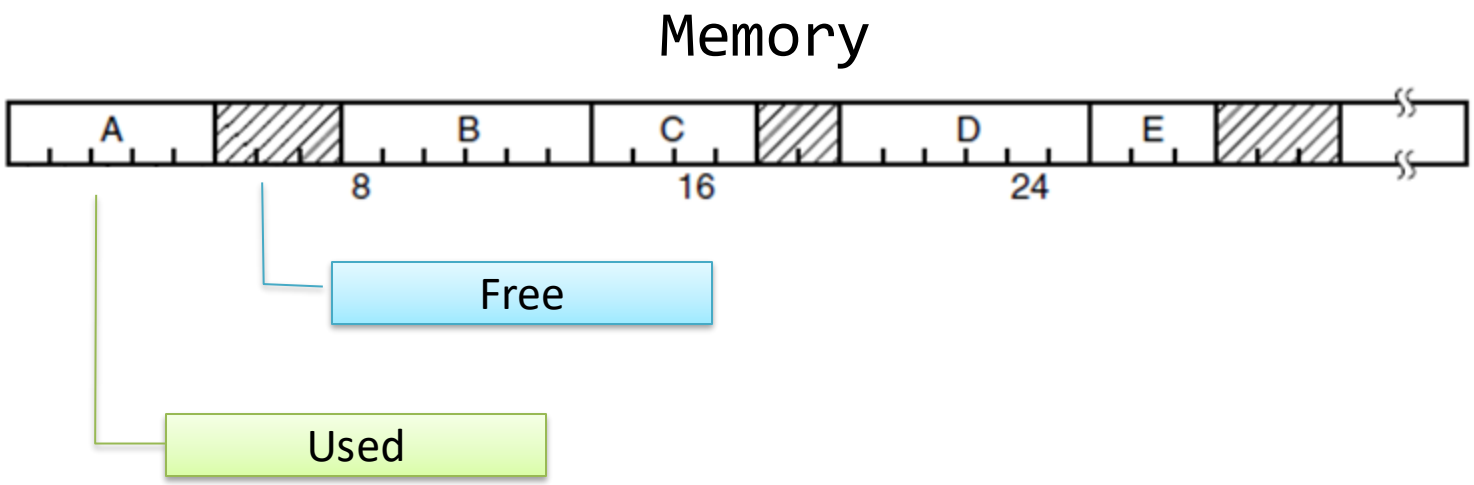


Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Managing Free Memory
Dynamic memory allocation

Two ways to keep track of memory usage:
Bitmaps and Link Lists

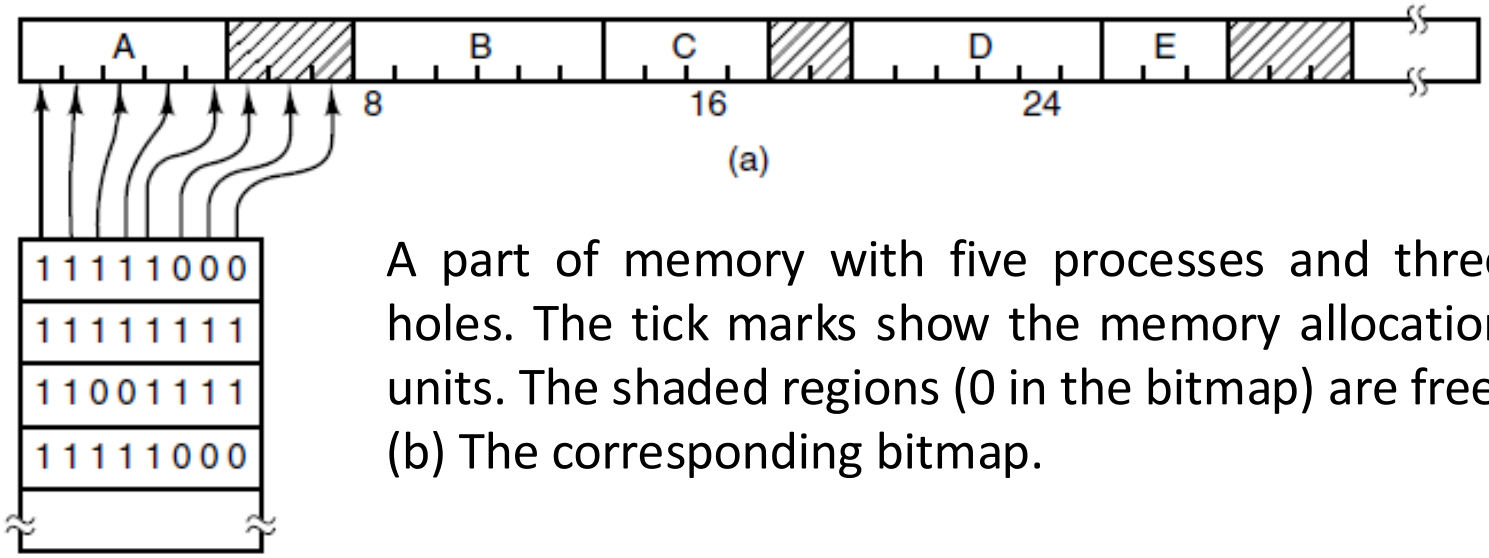


Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Bitmaps

Memory is divided into allocation units (e.g. several kilobytes).

A bit in the bitmap is 0 if the unit is free and 1 if it is occupied, e.g. K memory units results k map bits.



A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap.

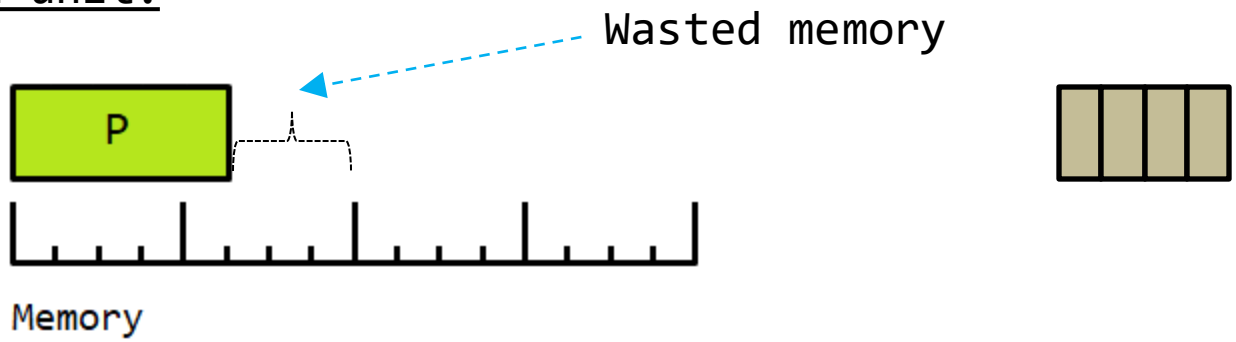
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Bitmaps

The size of the allocation unit is an important design issue.

- The smaller the allocation unit, the larger the bitmap.

- If the allocation unit is chosen large, the bitmap will be smaller, but appreciable memory may be wasted in the last unit of the process if the process size is not an exact multiple of the allocation unit.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

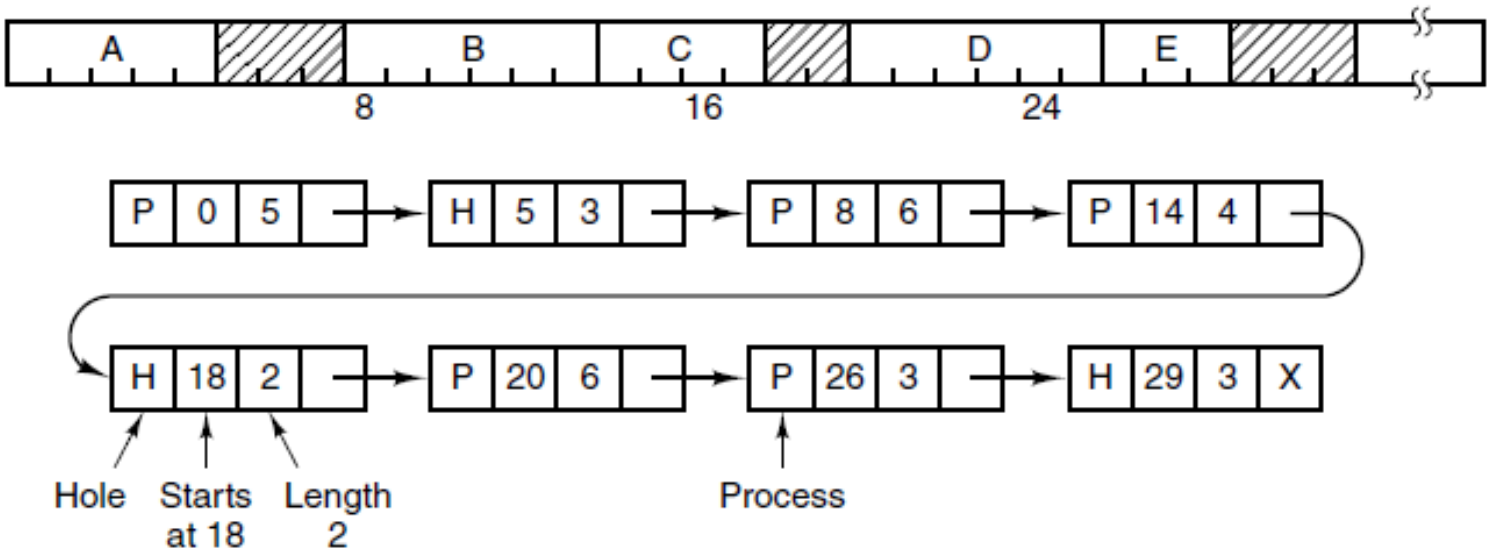
Memory Management with Bitmaps

To bring a *k*-unit process into memory, the memory manager must search the bitmap to find a run of *k* consecutive 0 bits in the map. Searching a bitmap is a slow operation.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Link List

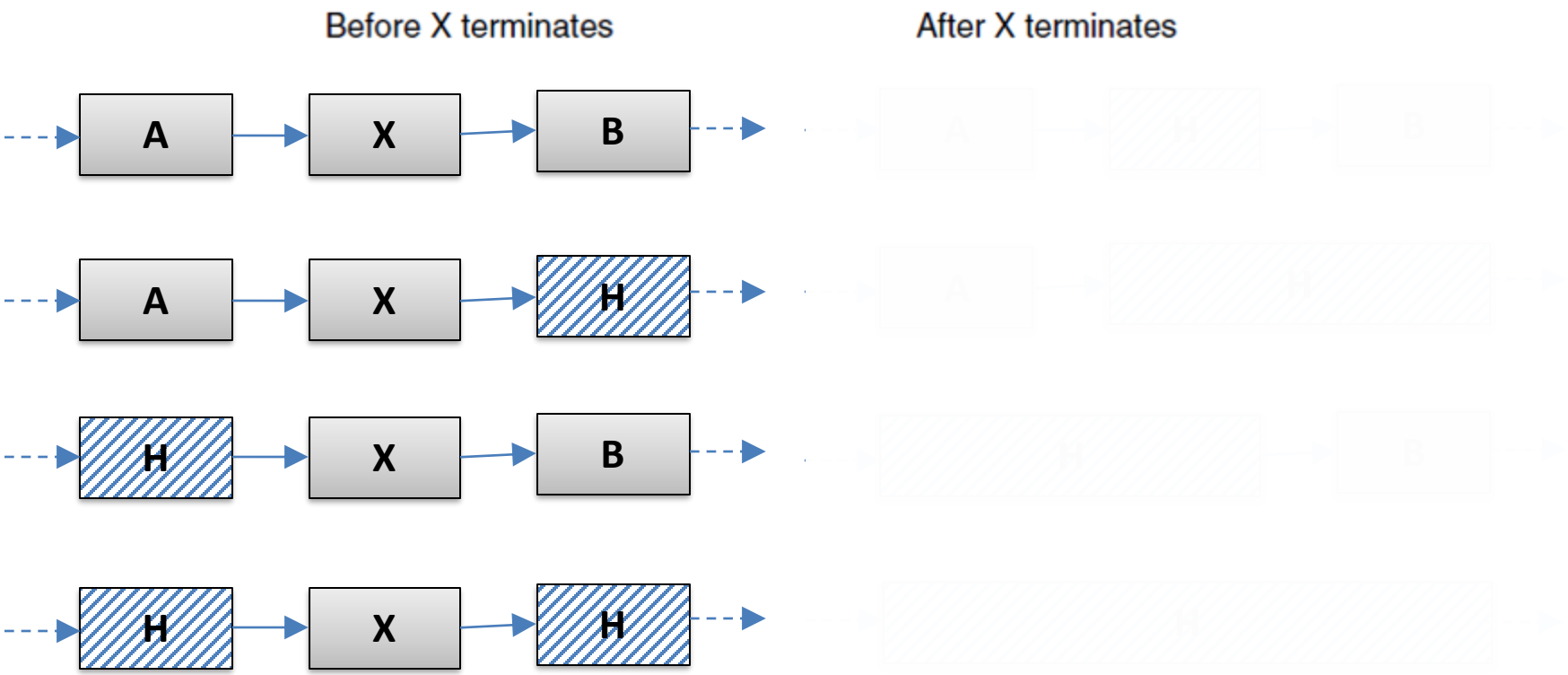
A linked list is used to keep track of memory. Each entry in the list specifies a hole (H) or process (P), the address at which it starts, the length, and a pointer to the next item.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Link List

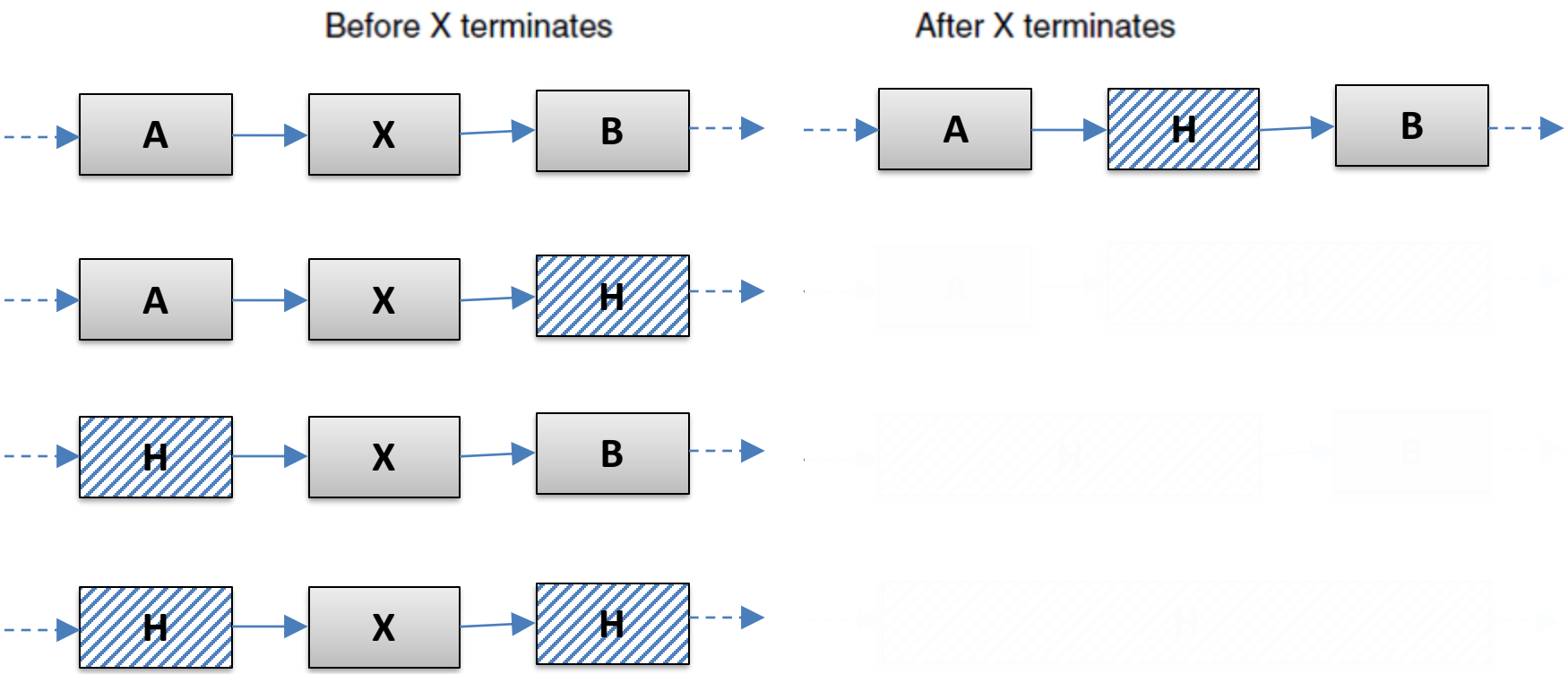
Four neighbor combinations for the terminating process, *X*.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Link List

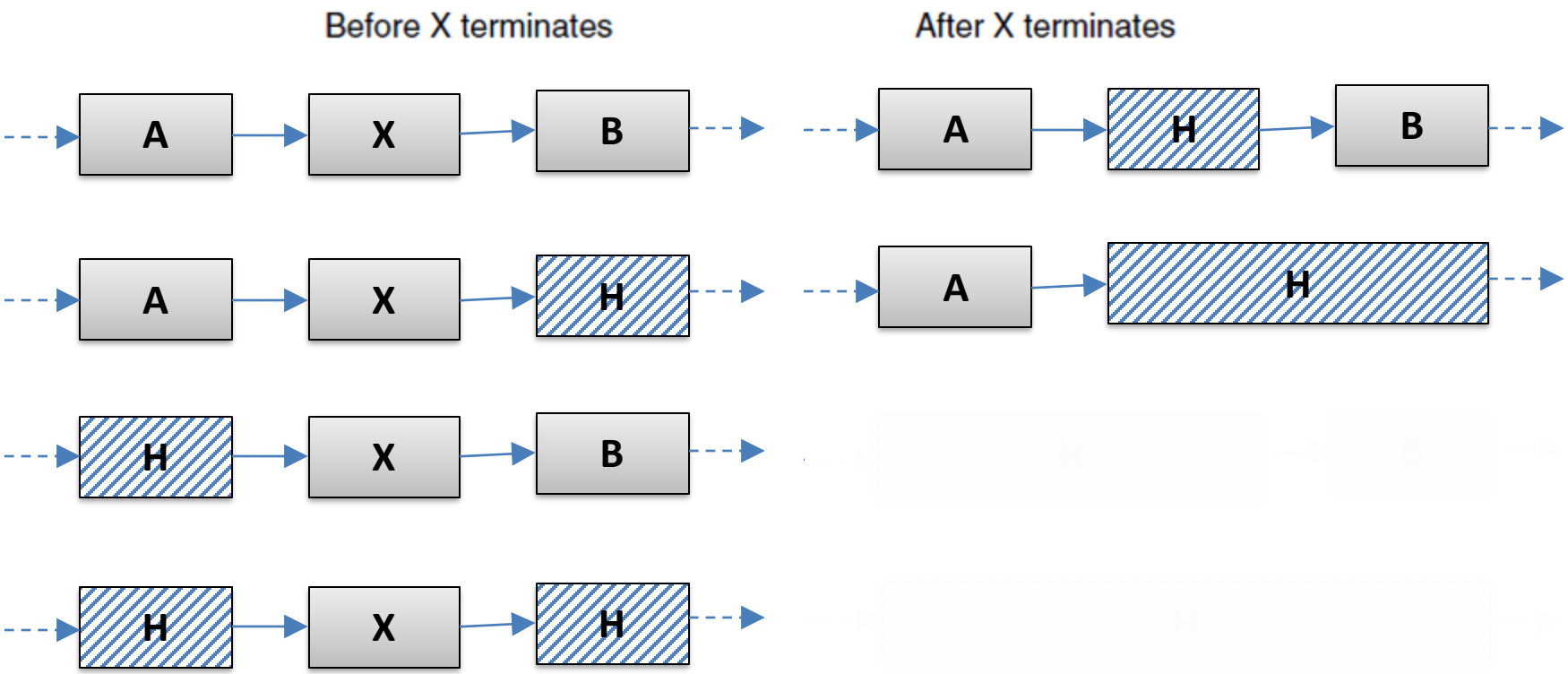
Four neighbor combinations for the terminating process, *X*.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Link List

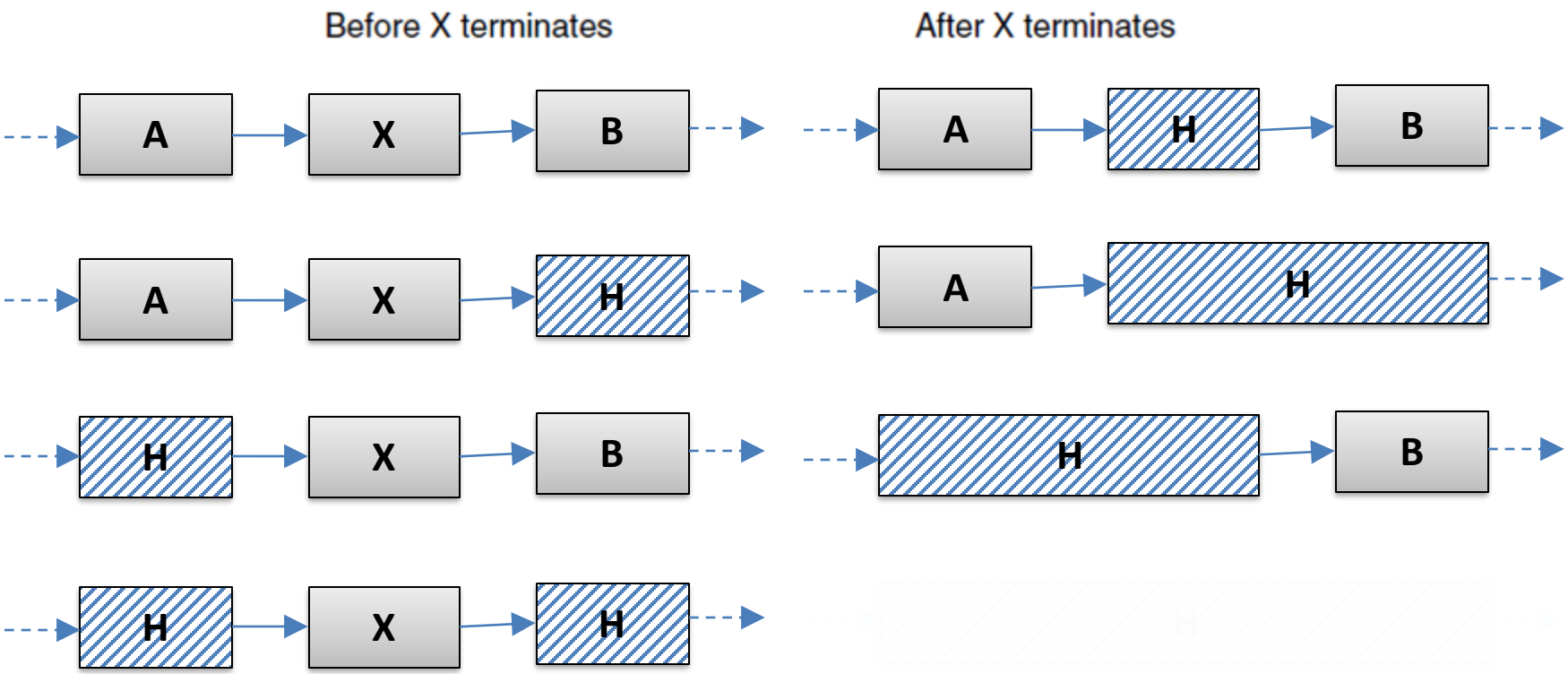
Four neighbor combinations for the terminating process, *X*.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Link List

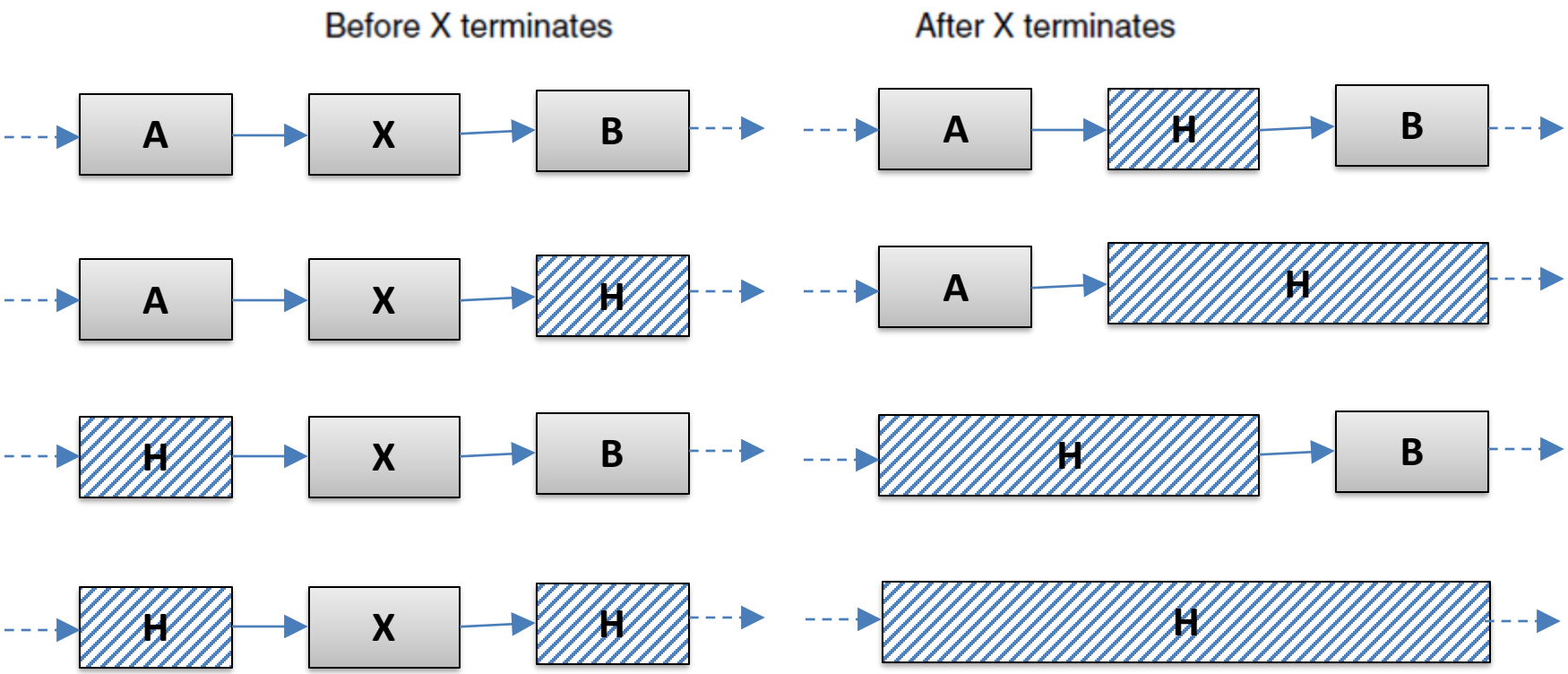
Four neighbor combinations for the terminating process, *X*.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management with Link List

Four neighbor combinations for the terminating process, *X*.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms

When a sorted list by address is used to keep track of memory. several algorithms can be used to allocate memory for a created process.

- **First fit**
- **Next fit**
- **Best fit**
- **Worst fit**
- **Quick fit**

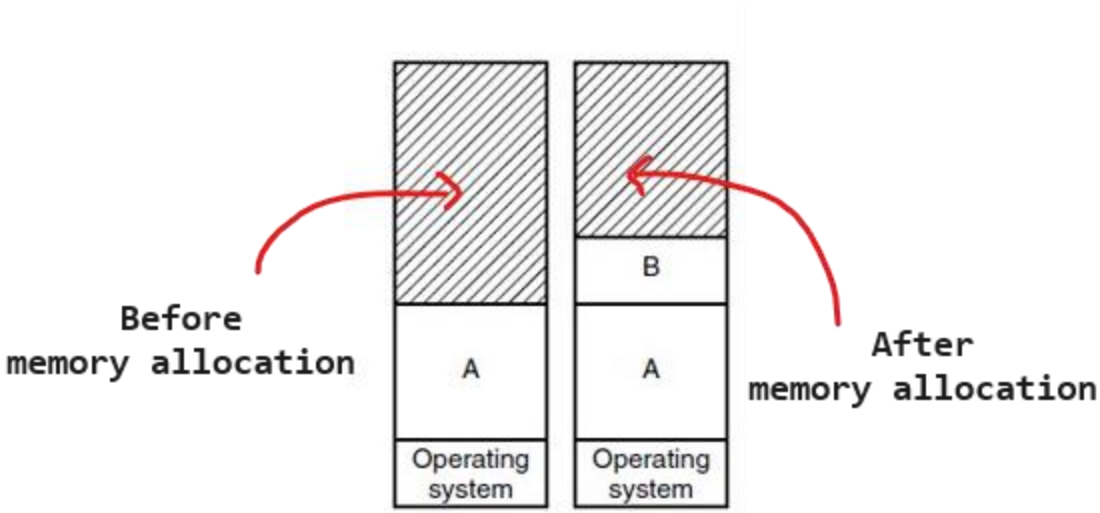
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms

First fit

- The size of allocated memory is known.
- The memory manager scans along the list of segments until it finds a hole that is big enough.
- The hole is then broken up into two pieces, one for the process and one for the unused memory.

First fit is a fast algorithm because it searches as little as possible.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms

Next fit

- It keeps track of where it is whenever it finds a suitable hole.
- The next time, it starts searching the list from the place where it left off last time, instead of always at the beginning, as first fit does.

Next fit gives slightly worse performance than first fit.

Best fit

Best fit searches the entire list, from beginning to end, and takes the smallest hole that is adequate (close to the actual size).

Best fit is slower than first fit because it must search the entire list every time it is called. It tends to fill up memory with tiny, useless holes.

Worst fit

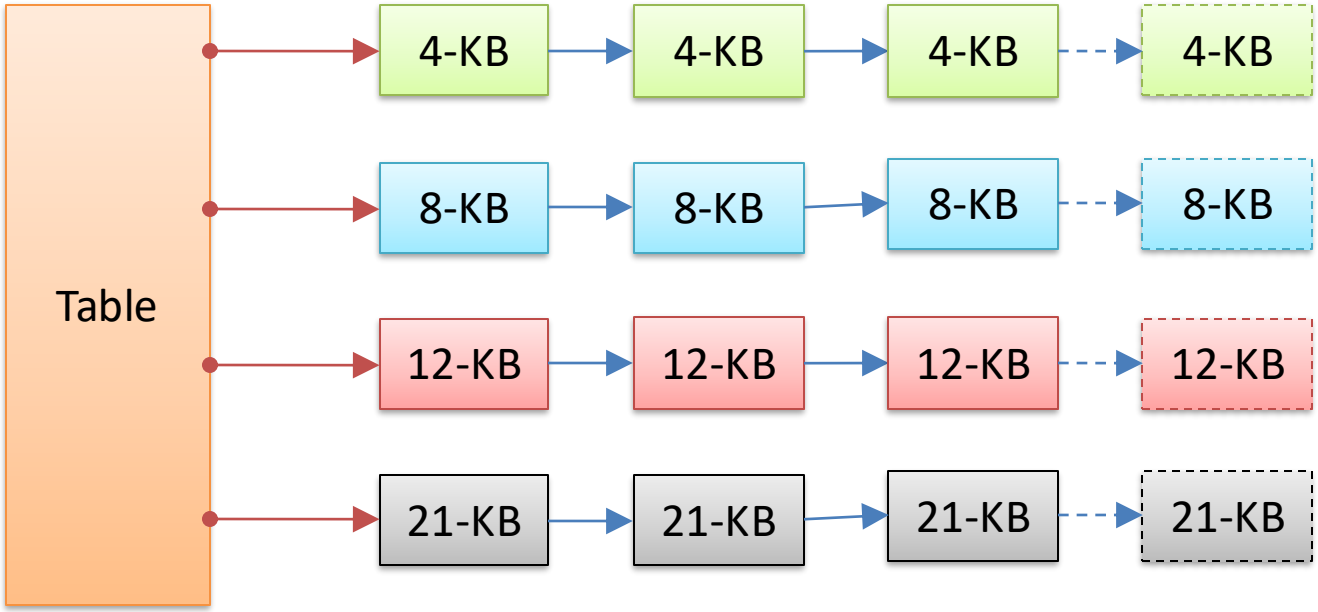
Worst fit takes the largest available hole, so that the new hole will be big enough to be useful.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms

Quick fit

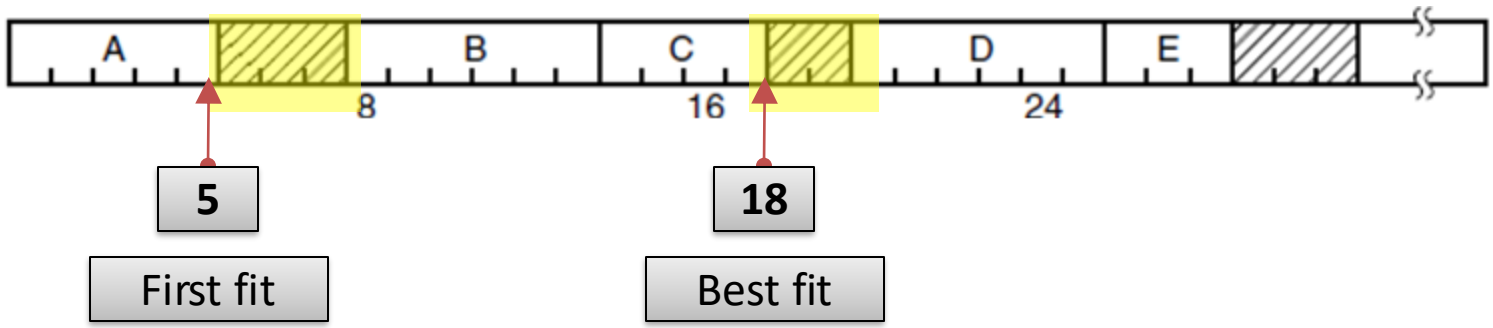
Quick fit maintains separate lists for some of the more common sizes requested.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms

As an example of first fit and best fit, consider the following figure again. If a block of size 2 is needed, first fit will allocate the hole at 5, but best fit will allocate the hole at 18.



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms
Buddy system

1. The memory is divided into a large block, typically a power of two in size. For example, if the system has 1 GB of memory, this could be considered a single block of size 1 GB.

2. When a request for memory is made, the system checks if a block of the requested size is available. If a block of the exact size is not available, the system splits the nearest larger block into two smaller "buddy" blocks, each half the size of the original. This splitting continues until the block size matches the request.

The "**buddies**" are two blocks that are halves of the same original block. They are always of the same size, and they are contiguous in memory. For example, if a 256 KB block is split, it produces two 128 KB buddy blocks.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms
Buddy system (Allocation and Deallocation)

The memory manager allocates the smallest block that is large enough to fulfill the request. When a block is freed, the system checks if its buddy block is also free. If both are free, they are combined (coalesced) back into a larger block. This process of merging continues up the chain until no further merging is possible.

By always splitting and merging in powers of two, the buddy system helps minimize fragmentation.

Internal fragmentation can still occur because allocated blocks are always powers of two, so if a request doesn't exactly match a power of two, some space might be wasted.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Memory Management Algorithms
Buddy system (Allocation and Deallocation)

Example:

Consider a system that uses the buddy system with a physical address space of 512 KB. Calculate the partition size required for a 25 KB process followed by a 126 KB process.

```
Original 512 KB
   |
   ├── 256 KB (free)
   └── 256 KB
       |
       ├── 128 KB (allocated to 126 KB process)
       └── 128 KB
           |
           ├── 64 KB (free)
           └── 64 KB
               |
               ├── 32 KB (allocated to 25 KB process)
               └── 32 KB (free)
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Virtual Memory

Motivations:

- There is a need to run programs that are too large to fit in memory.
- Swapping is a slow process(why?).

Solution:

Solution adopted in the 1960s, split programs into little pieces, called overlay kept on the disk, swapped in and out of memory.

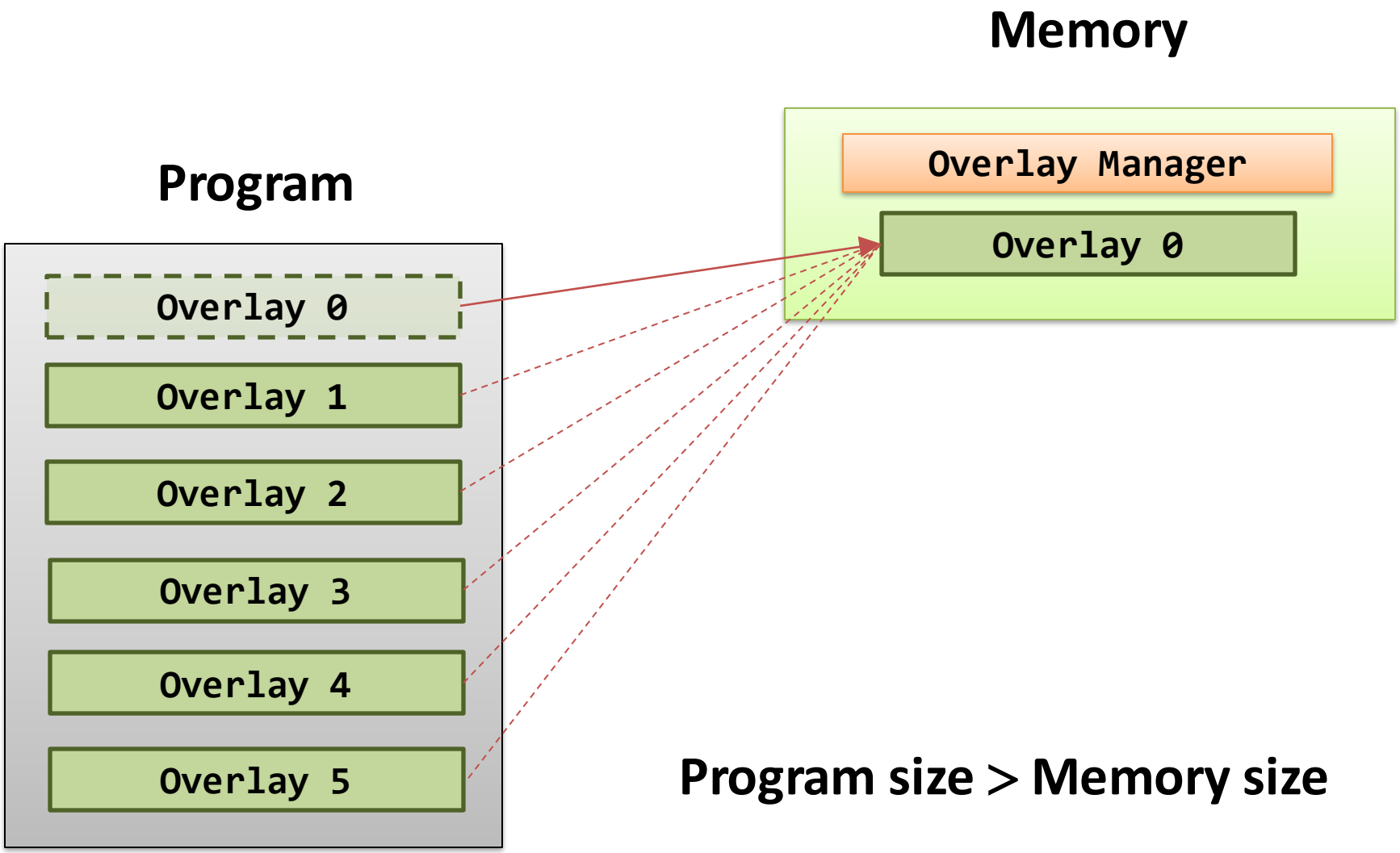
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Virtual Memory

Splitting the program into pieces had to be done manually by the programmer and was time consuming, boring, and error prone.

It did not take long before someone thought of a way to turn the whole job over to the computer.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.



Program size > Memory size

MS-DOS

DOS overlay files

```
C:\NU>dir *.ovl

 Volume in drive C is MS-DOS_6
 Volume Serial Number is 502B-58AE
 Directory of C:\NU

NORTON   OVL        68,101 06-01-92   6:01a
NDOS     OVL        73,526 06-01-92   6:01a
NUCONFIG OVL           876 06-01-92   6:01a
        3 file(s)        142,503 bytes
                     132,911,104 bytes free

C:\NU>_
```

The Norton Utilities 6.01

File name: NUCONFIG.OVL
File type: binary file
File size: 876 Bytes
File date: 6/1/1992, 6:01:00 AM

Virtual Memory

Splitting the program into pieces had to be done manually by the programmer and was time consuming, boring, and error prone.

It did not take long before someone thought of a way to turn the whole job over to the computer.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Virtual Memory

File Edit Sketch Tools Help

Arduino Uno

rand.ino

```
long randomNumber;
void setup() {
  Serial.begin(9600);
  randomSeed(42);
}
void loop() {
  randomNumber = random(2,500);
  delay(1000);
  Serial.print("The Random Number is = ");
  Serial.println(randomNumber);
}
```

core	2/14/2023 11:25 PM	File folder	
libraries	2/14/2023 11:25 PM	File folder	
preproc	2/14/2023 11:25 PM	File folder	
sketch	2/14/2023 11:41 PM	File folder	
build.options.json	2/14/2023 11:41 PM	JSON Source File	1 KB
compile_commands.json	2/14/2023 11:41 PM	JSON Source File	2 KB
includes.cache	2/14/2023 11:41 PM	CACHE File	1 KB
libraries.cache	2/14/2023 11:41 PM	CACHE File	1 KB
rand.ino.eep	2/14/2023 11:41 PM	Atmel EEPROM co...	1 KB
rand.ino.elf	2/14/2023 11:41 PM	ELF File	28 KB
rand.ino.hex	2/14/2023 11:41 PM	Intel HEX file	7 KB
rand.ino.with_bootloader.bin	2/14/2023 11:41 PM	Binary file	32 KB
rand.ino.with_bootloader.hex	2/14/2023 11:41 PM	Intel HEX file	8 KB

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Virtual Memory

Start code | Byte count | Address | Record type | Data | Checksum

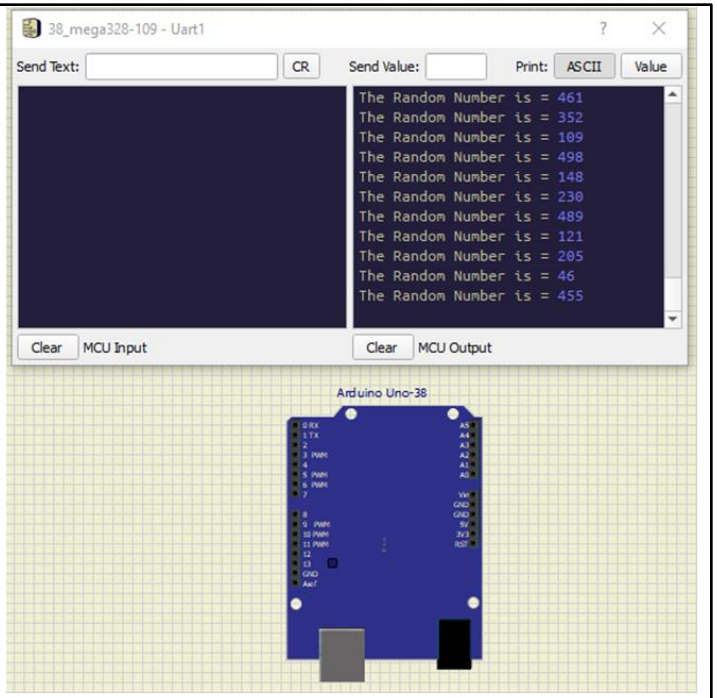
```
:100100002146013601214701360 07EFE09D2190140
:100110002146017E17C20001FF5F16002148011928
:10012000194E79234623965778239EDA3F01B2CAA7
:100130003F0156702B5E712B722B732146013421C7
:00000001FF
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Virtual Memory

Start code	Byte count	Address	Record type	Data	Checksum
:	10	0000	00	0C9434000C9446000C9446000C944600	6A

```
:100000000C9435000C945D000C945D000C945D0024
:100010000C945D000C945D000C945D000C945D00EC
:100020000C945D000C945D000C945D000C945D00DC
:100030000C945D000C945D000C945D000C945D00CC
:100040000C9479010C945D000C94E9010C94C301AB
:100050000C945D000C945D000C945D000C945D00AC
:100060000C945D000C945D001C0311241FBECFEFA7
:10007000D8E0DEBFCDBF11E0A0E0B1E0E0EBF8E0FA
:1000800002C005900D92A233B107D9F721E0A2E397
                    .
                    .
                    .
:10092000911DB29F700D811D1124911D0895EE0F30
:0E093000FF1F0590F491E02D0994F894FFCF7D
:10093E00010000000000000024018400B1007101DC
:10094E00E200C000D400546865205261 6E646F6D81
:10095E00204E756D626572206973203D20000D0A70
:02096E00000087
:00000001FF
```



Size = 147*16 + 14 + 3*16 + 2 = 2,416 bytes

```
Sketch uses 2416 bytes (7%) of program storage space. Maximum is 32256 bytes.
Global variables use 220 bytes (10%) of dynamic memory, leaving 1828 bytes for local variables. Maximum is 2048 bytes.
```

Virtual Memory

- Each process has its own address space, which is broken up into chunks called pages.
- The corresponding units in the physical memory are called page frames.
- The pages and page frames generally have the same size.
- Each page is a contiguous range of addresses.
- These pages are mapped onto physical memory, but not all pages have to be in physical memory at the same time to run the program.
- **M**emory **M**anagement **U**nit (hardware) maps the virtual addresses onto the physical memory addresses.
- The relation between virtual addresses and physical memory addresses is given by the page table.
- In a sense, virtual memory is a generalization of the base-and-limit-register idea.

Virtual Memory

The OS initially loads only essential parts of the process, called the "resident set". As the process executes, if it references memory outside the resident set, a memory access fault occurs, prompting the OS to load the necessary part from disk into memory. This allows multiple processes to reside in memory simultaneously, improving system efficiency by increasing the likelihood of having a process ready to run.

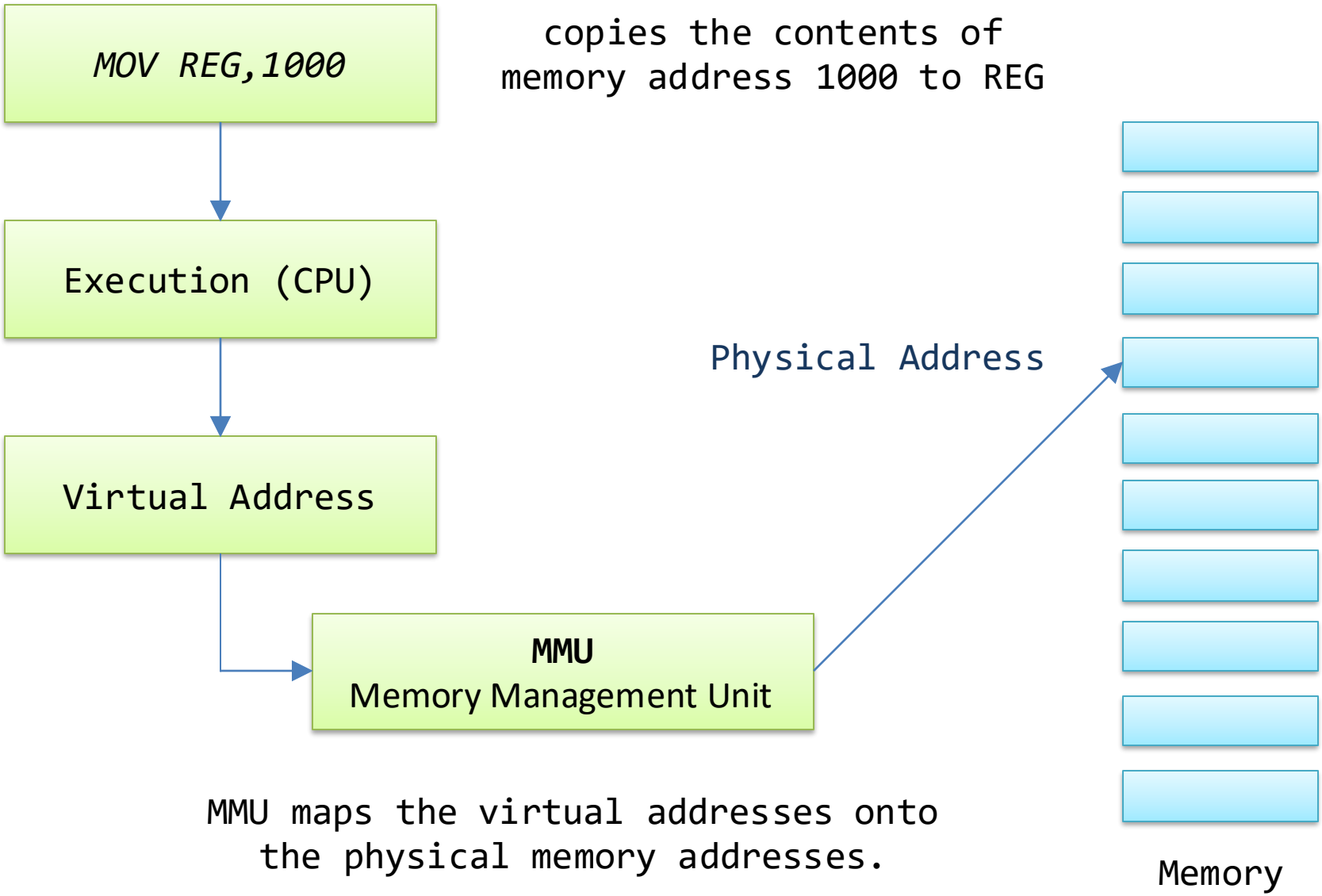
The two main advantages of this strategy are:

- **Increased Process Capacity:** More processes can be kept in main memory since only portions of each are loaded, leading to better processor utilization.

- **Handling Larger Processes:** A process can be larger than the available main memory, as the OS dynamically loads parts of it as needed. This removes the need for programmers to manually manage memory constraints, as virtual memory creates the illusion of a much larger memory space.

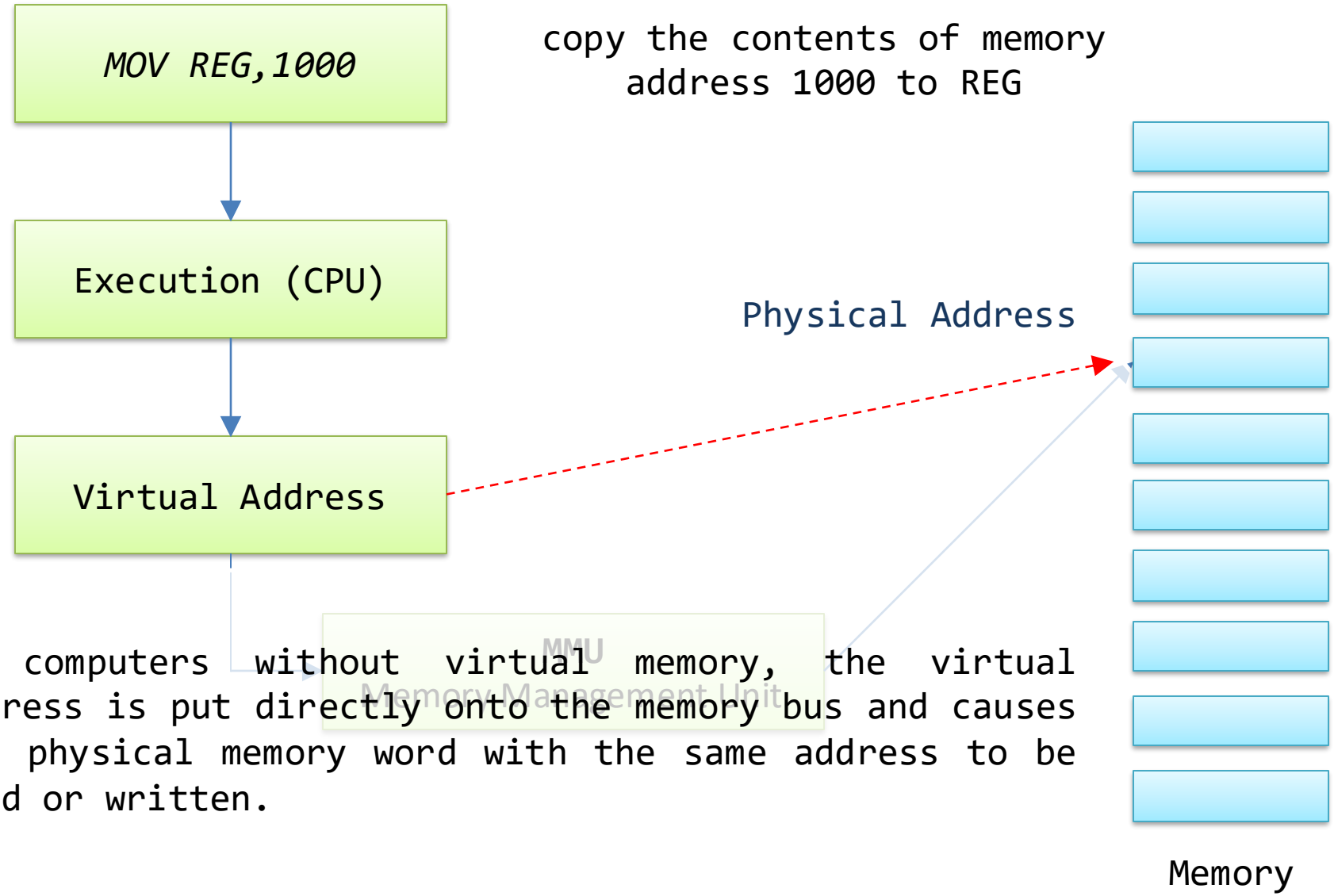
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Virtual Memory



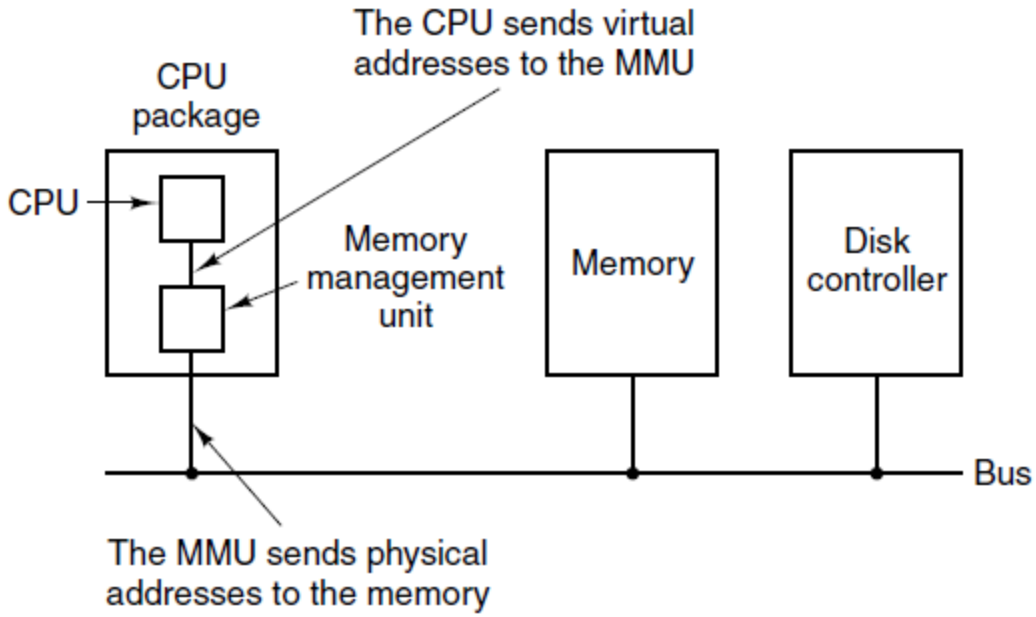
MMU maps the virtual addresses onto the physical memory addresses.

Virtual Memory



On computers without virtual memory, the virtual address is put directly onto the memory bus and causes the physical memory word with the same address to be read or written.

Paging

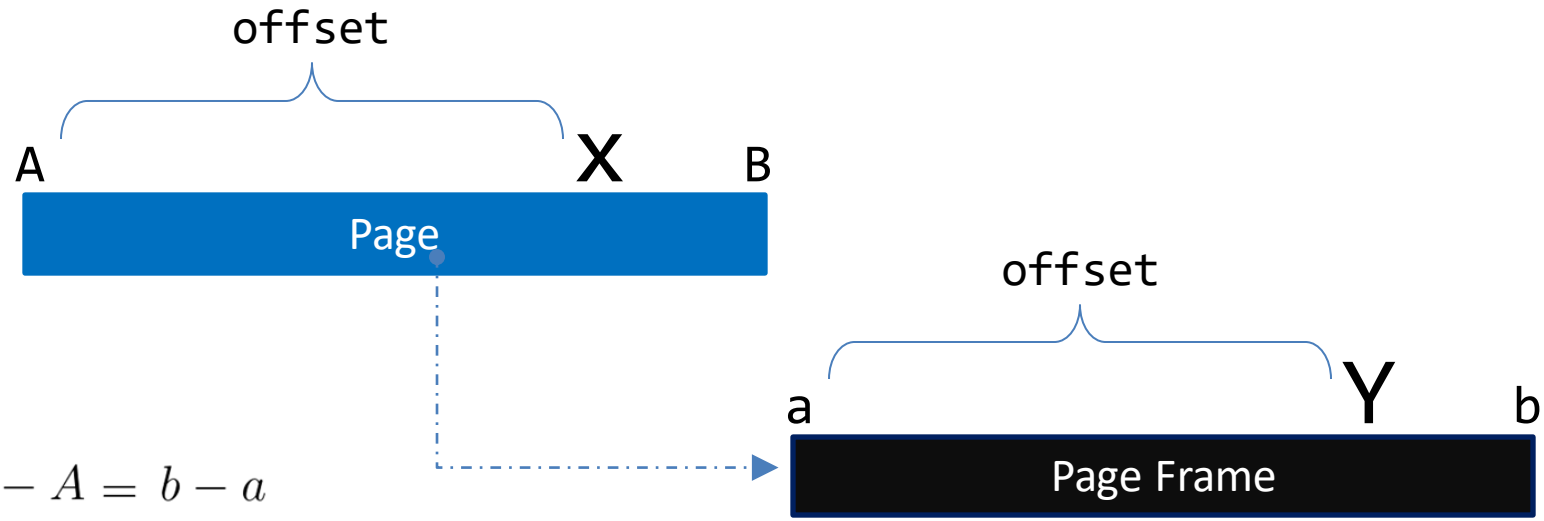


The position and function of the MMU. Here the MMU is shown as being a part of the CPU chip because it commonly is nowadays. However, logically it could be a separate chip and was years ago.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging

X (virtual address) = offset + A => offset = X - A



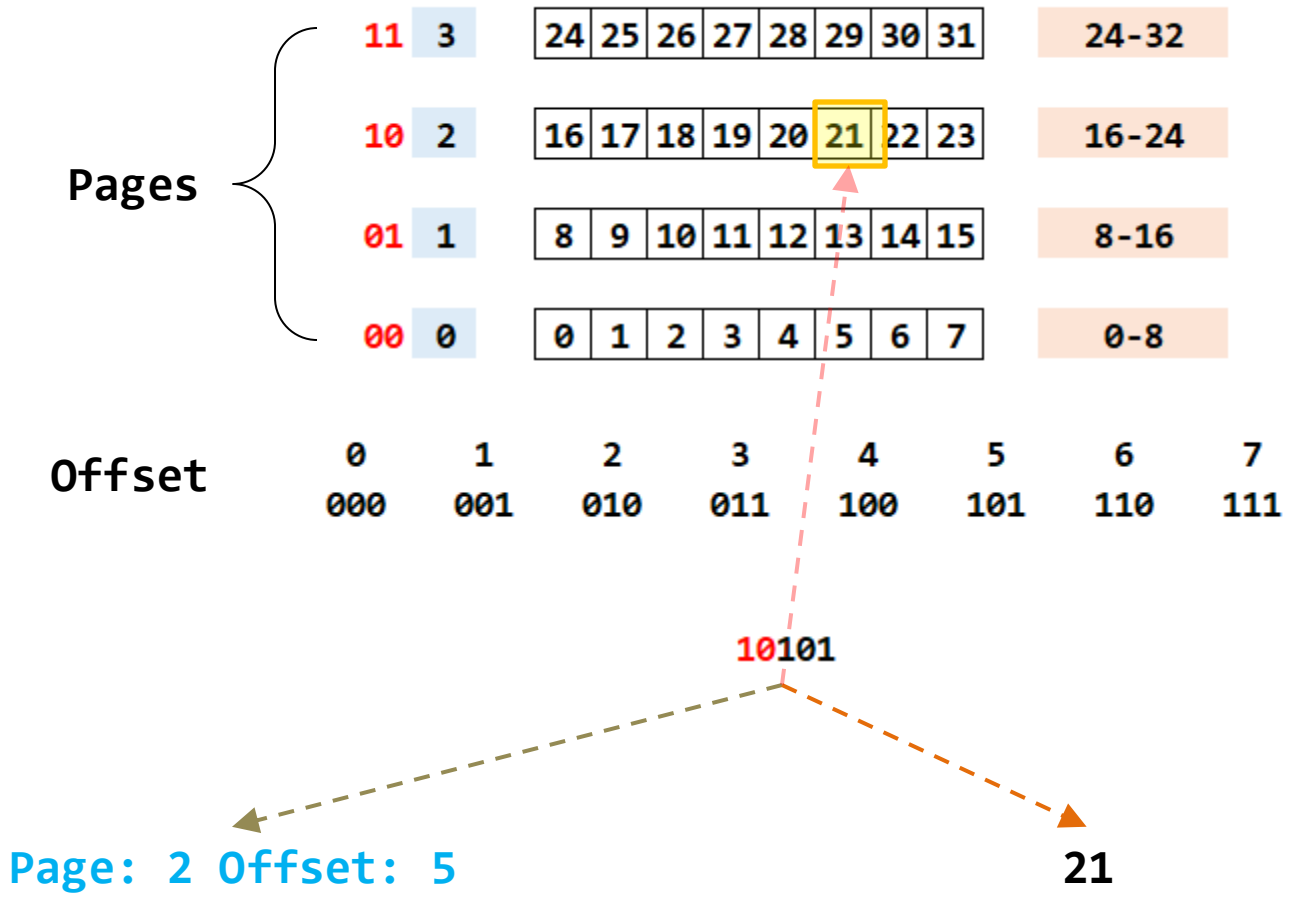
$$B - A = b - a$$

Y (physical address) = offset + a => offset = Y - a

offset = Y – a = X – A => Y – a = X – A

$$Y = X + a - A = X + b - B$$

Paging


Pages
11 3 24 25 26 27 28 29 30 31 24-32
10 2 16 17 18 19 20 21 22 23 16-24
01 1 8 9 10 11 12 13 14 15 8-16
00 0 0 1 2 3 4 5 6 7 0-8
Offset
0 1 2 3 4 5 6 7
000 001 010 011 100 101 110 111
10101
Page: 2 Offset: 5
21

Paging

Example:

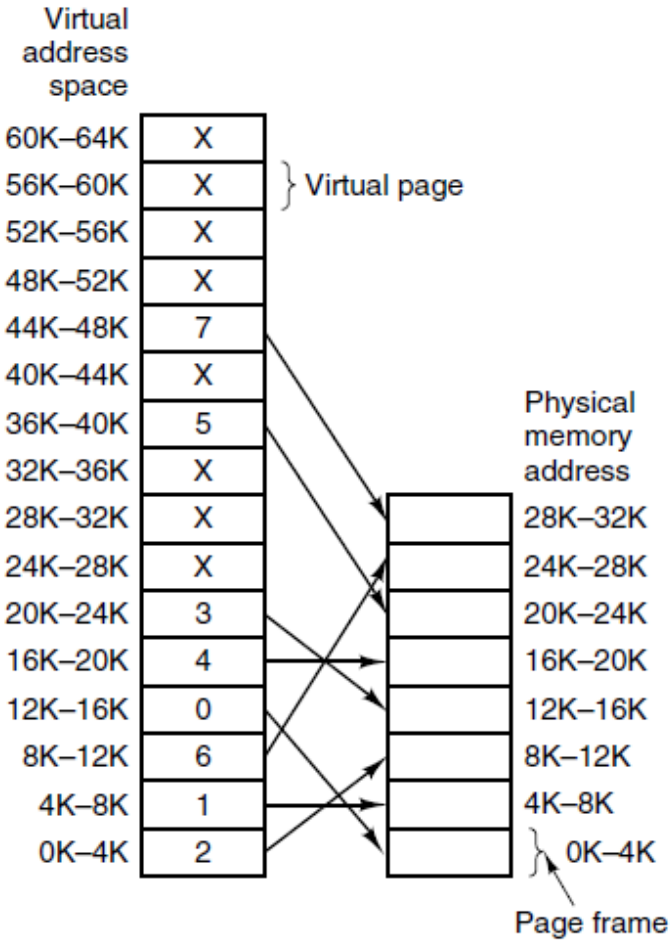
page size and page frame size = 4 KB

Virtual address 64 KB = 2^{16} (16 bits)

Physical memory 32 KB = 2^{15} (15 bits)

64 KB/4 KB = 16 pages

32 KB/4 KB = 8 page frames

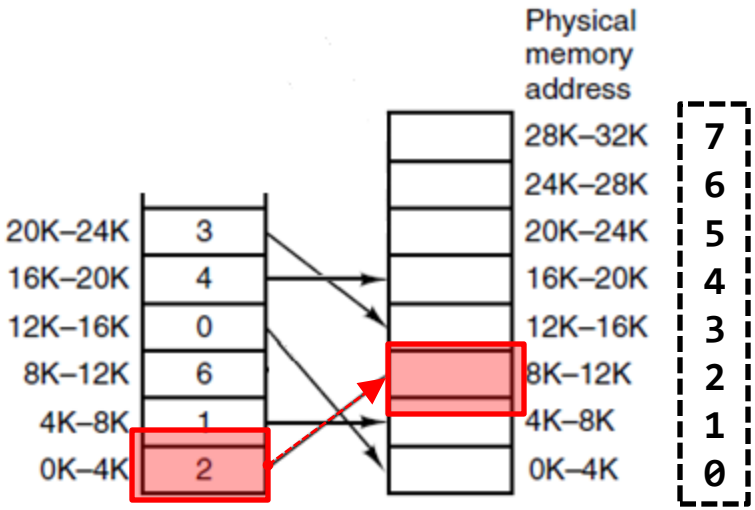


Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging

Example:

1) MOV REG, 0
2) MOV REG, 8192
3) MOV REG, 20500



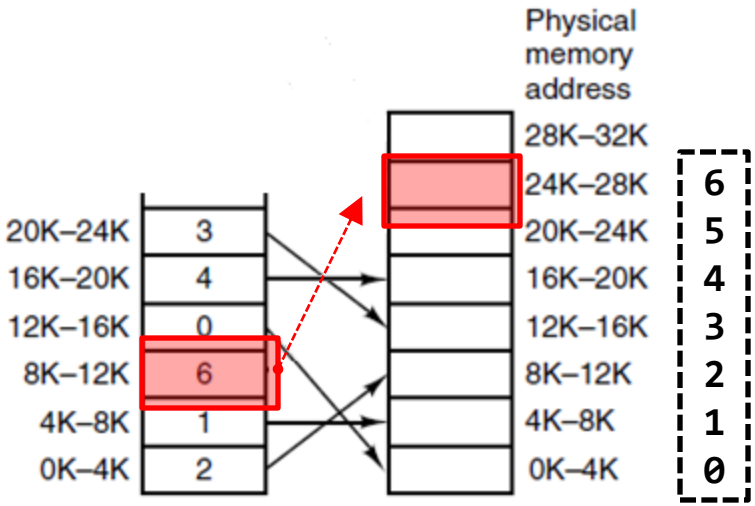

```
Virtual address = 0
Physical address = 8K + offset = 8*1024 + 0 = 8192
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging

Example:

1) MOV REG, 0
2) MOV REG, 8192
3) MOV REG, 20500



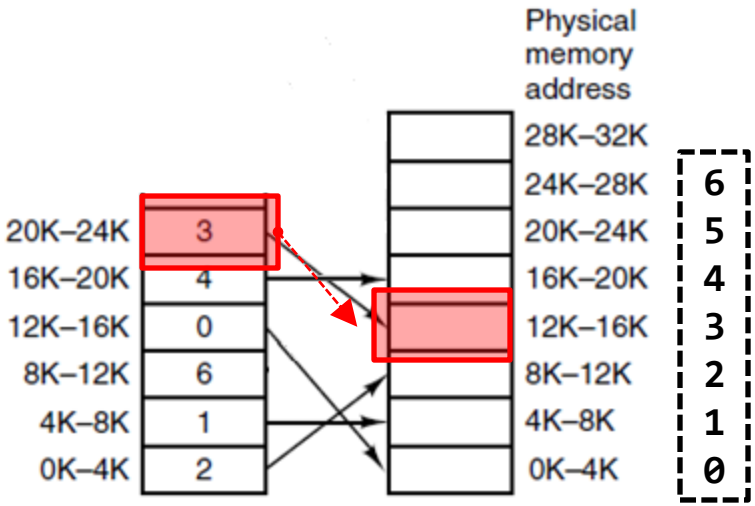

```
Virtual address = 8192
Offset = 8192 - 8K  = 8192 - 8*1024  = 0
Physical address = 24K + offset = 24*1024 + 0 = 24576
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging

Example:

1) MOV REG, 0
2) MOV REG, 8192
3) MOV REG, 20500



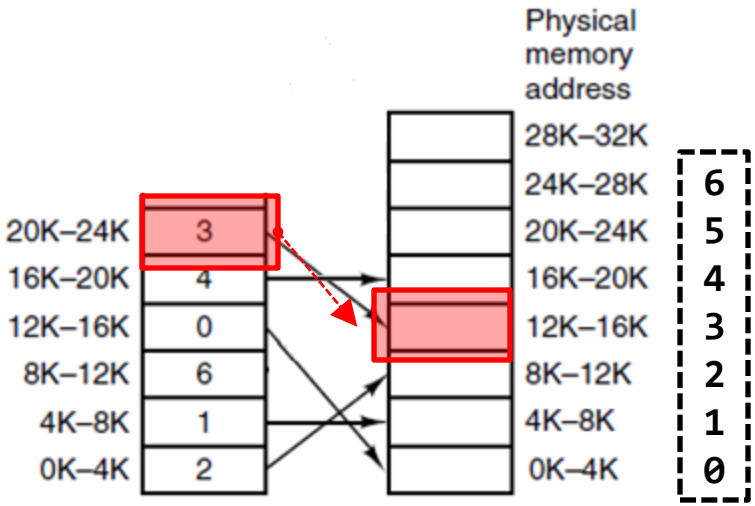

```
Virtual address = 20500
Offset = 20500 - 20K = 20500 - 20*1024 = 20
Physical address = 12K + offset = 12*1024 + 20 = 12308
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging

Example:

1) MOV REG, 0
2) MOV REG, 8192
3) MOV REG, 20500




```
A = 20*1024 = 20480
B = 24*1024 = 24576
a = 12*1024 = 12288
b = 16*1024 = 16384
x = 20500
Y = x + b - B = 20500 + (16384 - 24576) = 20500 – 8192 = 12308
Y = x + a - A = 20500 + (12288 - 20480) = 12308
```

$$B - A = b - a$$

$$Y = X + a - A = X + b - B$$

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Design Issues For Paging Systems

Page Size

In general, a large page size will cause more wasted space to be in memory than a small page size.

On the other hand, small pages mean that programs will need many pages, and thus a large page table and less internal fragmentation.

On the average, half of the final page will be empty (this wastage is called internal fragmentation).

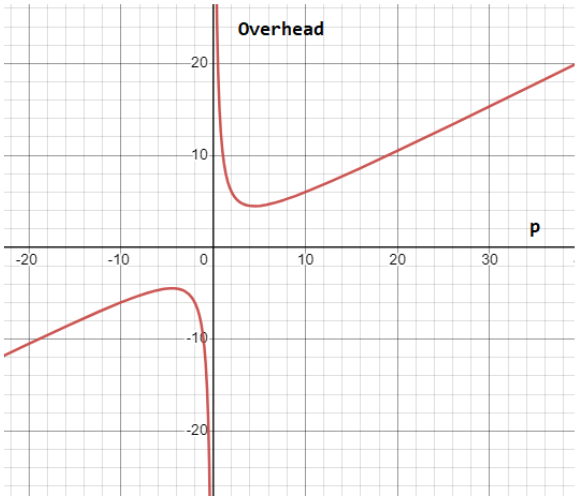
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Design Issues For Paging Systems

Which Page Size is Best?

Let
s = Average process size in bytes
p = Page size in bytes
e = Each page table entry requires in bytes

s/p = Number of pages per process
es/p = Size of page table
p/2 = space wasted due to internal fragmentation



$$\text{overhead} = \frac{se}{p} + \frac{p}{2}$$

$$\frac{d(\text{overhead})}{dp} = 0 \Rightarrow -\frac{se}{p^2} + \frac{1}{2} = 0 \quad \Rightarrow p = \sqrt{2se}$$

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Design Issues For Paging Systems

Example:
For s = 1MB and e = 8 bytes per page table entry, the optimum page size is:

$$p = \sqrt{2 \times 2^{20} \times 8} = \sqrt{2^{24}} = 2^{12} = 4 \text{ KB}$$

Commercially available computers have used page sizes ranging from 512 bytes to 64 KB. A typical value used to be 1 KB, but nowadays 4 KB is more common.

Many processors support a variety of page sizes that the operating system can choose from and combine as needed.

The x86-64 architecture supports 4 KB, 2 MB, and 1 GB pages, allowing the use of 4 KB pages for user applications and a single 1 GB page for the kernel.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Design Issues For Paging Systems
Linux

```
~$ getconf  PAGE_SIZE
4096
~$ getconf  PAGESIZE
4096
~$ getconf  -a | grep PAGESIZE
PAGESIZE                           4096
~$ getconf  -a | grep PAGE_SIZE
PAGE_SIZE                          4096
```

```
GETPAGESIZE(2)
NAME
       getpagesize - get memory page size
SYNOPSIS
       #include <unistd.h>
       int getpagesize(void);
```

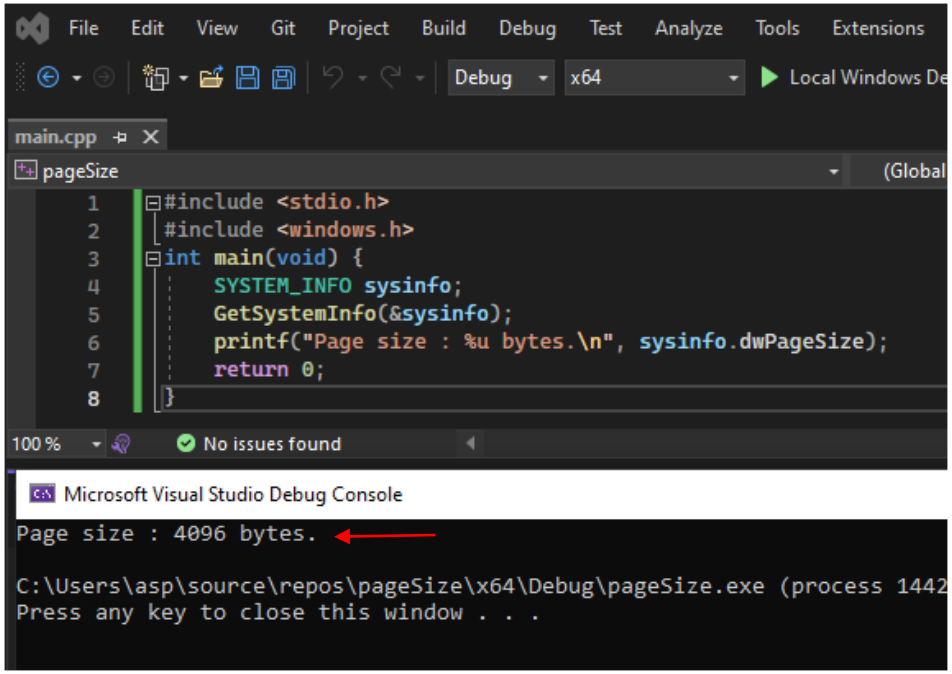
```c
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("page size : %d ", getpagesize());
    return 0;
}

```

```
page size : 4096
```

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Design Issues For Paging Systems Windows

```c
#include <stdio.h>
#include <windows.h>
int main(void) {
    SYSTEM_INFO sysinfo;
    GetSystemInfo(&sysinfo);
    printf("Page size : %u bytes.\n", sysinfo.dwPageSize);
    return 0;
}
```



Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging
Page Fault

When the program references an unmapped address to access a block of memory that is not stored in the physical memory, the MMU causes the CPU to trap to the operating system. This trap is called a page fault.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging
Page Fault

- **Page Fault Occurs**:
  The CPU generates a page fault interrupt when a process tries to access a page that is not currently in physical memory.

- **Interrupt Handling**:
  The CPU transfers control to the operating system's page fault handler.

- **Save State**:
  The operating system saves the state of the process that caused the page fault (CPU registers, program counter, etc.).

- **Determine Faulting Page**:
  The OS identifies which virtual page caused the fault and determines the necessary action.

- **Check Physical Memory**:
  The OS checks if there is a free physical frame available. If not, it selects a page to swap out (if needed).

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging
Page Fault

- **Swap Out Page:**
  If necessary, the OS writes the selected page to disk (swap space) to free up a physical frame.

- **Load Required Page:**
  The OS loads the required page from disk into the now-available physical frame.

- **Update Page Table:**
  The OS updates the page table to reflect the new mapping of the virtual page to the physical frame.

- **Restore Process State:**
  The OS restores the state of the interrupted process and updates the program counter to resume execution.

- **Resume Execution:**
  The process continues execution from the point where it was interrupted.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Page Replacement Algorithms

When a page fault happens, the operating system needs to remove a page from memory to make space for the new one. If the removed page has been modified, it is written to disk to update its stored copy. If it hasn't been modified, no disk write is necessary, and the new page directly replaces the evicted one in memory.

The primary challenge is deciding which memory frame to replace when a page fault occurs.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Page Replacement Algorithms

- Optimal algorithm
- Not recently used algorithm
- First-in, first-out (FIFO) algorithm
- Second-chance algorithm
- Clock algorithm
- Least recently used (LRU) algorithm
- Working set algorithm
- WSClock algorithm

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging
How does MMU work?

I. The virtual address is split into a virtual page number (high-order bits) and an offset (low-order bits).

II. The virtual page number is used as an index into the page table to find the entry for that virtual page.

III. From the page table entry, the page frame number (if any) is found.

IV. The page frame number is attached to the high-order end of the offset, replacing the virtual page number, to form a physical address that can be sent to the memory.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging
How does MMU work?

Example:

The virtual address 8196 (0010000000000100 in binary) is mapped using the MMU map as illustrated in the following figure.

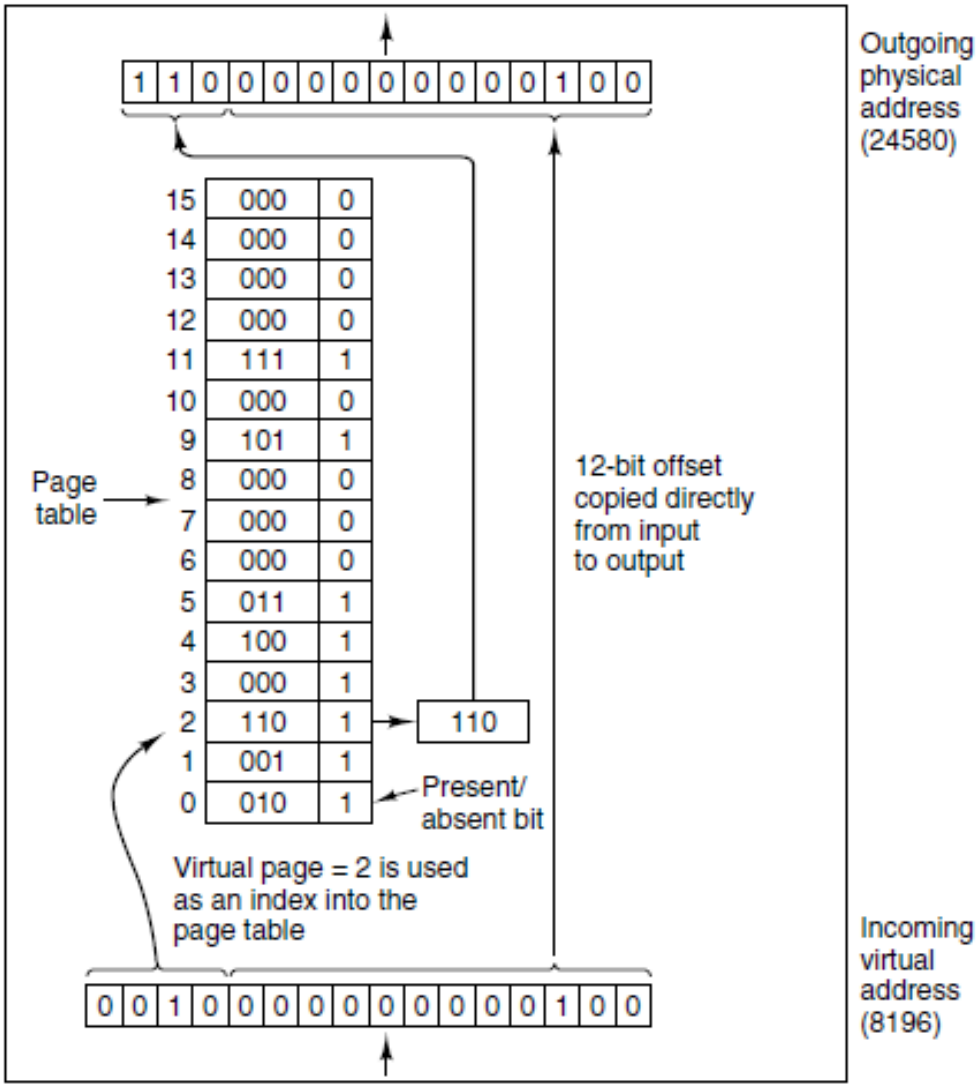
The incoming 16-bit virtual address is split into a 4-bit page number and a 12-bit offset. With 4 bits for the page number, we can have 16 pages, and with 12 bits for the offset, we can address all 4096 bytes within a page.

(Virtual address) 0010000000000100 ⇒ 110000000000100 (physical memory address)

Page number Frame page number

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Paging
How does MMU work?



The internal operation of the MMU with 16 4-KB pages.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Structure of a Page Table Entry

A sample page table entry:

Size. The size varies from computer to computer, but 32 bits is a common size. <u>The most important field is the Page frame number</u>.

Present/absent bit. If this bit is 1, the entry is valid and can be used. Accessing a page table entry with this bit set to 0 causes a page fault.

Protection bits. The Protection bits tell what kinds of access are permitted.

1-bit :

- 0 read/write
- 1 read only

3-bits:

- 1 bit for each rwx (reading writing executing)

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Structure of a Page Table Entry

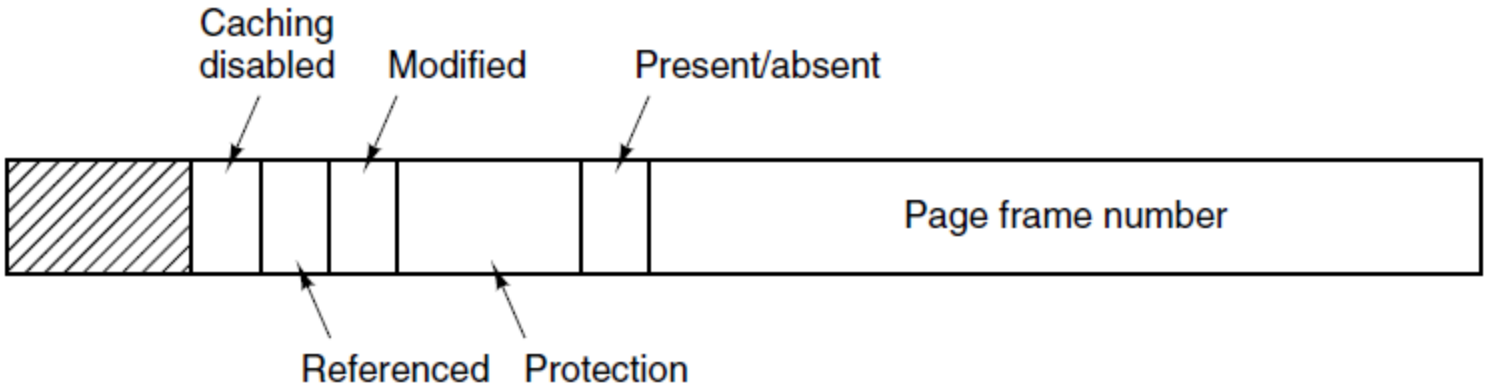
Modified bit. The modified bit keep track of page usage. If the page in it has been modified (i.e., is "**dirty**"), it must be written back to the disk. If it has not been modified (i.e., is "clean"), it can just be abandoned, since the disk copy is still valid.

Referenced bit. The Referenced bit is set whenever a page is referenced, either for reading or for writing.

Caching disabled. The Caching disabled bit allows caching to be disabled for the page.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Structure of a Page Table Entry



A typical page table entry.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Speeding Up Paging

Major issues faced:

1. The mapping from virtual address to physical address must be fast.

2. If the virtual address space is large, the page table will be large.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Speeding Up Paging

Paging issues

Based on the observation, most programs tend to make a large number of references to a small number of pages. Thus only a small fraction of the page table entries are heavily read; the rest are barely used at all.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Speeding Up Paging

Paging issues

Computers are equipped with a small hardware device for mapping virtual addresses to physical addresses <u>without going through the page table</u>.

The device, called a TLB (Translation Lookaside Buffer) or sometimes an <u>associative memory</u>.

It is usually inside the MMU and consists of a small number of entries, eight in this example, but rarely more than 256.

Speeding Up Paging

Each entry contains following information.

- Virtual page number.
- Modification bit. This bit indicates whether the page is modified or not.
- Protection code. read/write/execute permissions
- Validation bit. This bit indicates whether the entry is valid (i.e., in use) or not.

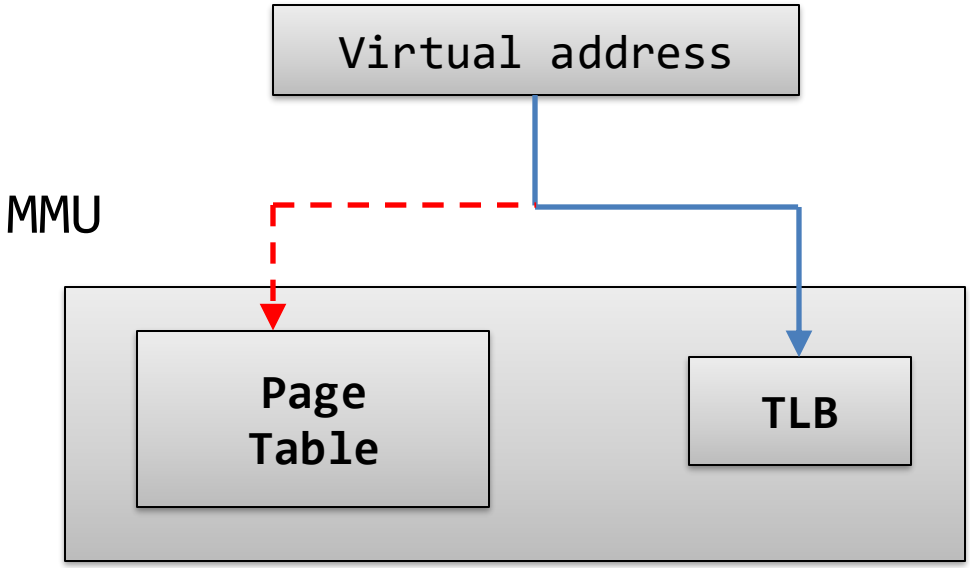
Translation Lookaside Buffers TLB

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

A TLB to speed up paging.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Translation Lookaside Buffers
How TLB functions



Valid match: the page frame is taken directly from the TLB.

When the virtual page number is not in the TLB. The MMU detects the miss and does an ordinary page table lookup.

Translation Lookaside Buffers
Soft Miss and Hard Miss

Soft miss:
The page referenced is not in the TLB, but is in memory. Updating the TLB is all that's required in this case; no disk or SSD I/O is necessary.

Hard miss:
A hard miss happens when the page is neither in memory nor in the TLB, necessitating a disk or SSD access to load the page into memory.

Translation Lookaside Buffers
Soft Miss and Hard Miss

Since TLB entries are limited and crucial for performance, using large pages wherever possible is beneficial.

To optimize these tradeoffs, operating systems may use different page sizes in various parts of the system, such as employing large pages for the kernel and smaller pages for user processes.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Translation Lookaside Buffers
Software TLB Management

On some RISC machines like SPARC, MIPS, and the now-defunct HP PA, page management is handled by software rather than the MMU.

When a TLB miss occurs, these systems generate a TLB fault and rely on the operating system to resolve it. The OS must locate the page, update the TLB by removing an old entry and adding a new one, and then restart the instruction that caused the fault.

This process must be efficient, as TLB misses are more common than page faults.

Page Tables for Large Memories

Example :

Virtual address : 32 bits

Page size : 4KB = 2^{12}
Number of pages : $2^{32}/2^{12} = 2^{20}$ = 1M page table entries

Page size : 1M = 2^{20}
Number of pages : $2^{32}/2^{20} = 2^{12}$ = 4K page table entries

Virtual address : 64 bits

Page size : 16M = 2^{24}
Number of pages : $2^{64}/2^{24} = 2^{40}$ = 1024G page table entries

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Page Tables for Large Memories

TLBs can be used to speed up virtual-to-physical address translation over the original page-table-in-memory scheme.

Another problem is how to deal with very large virtual address spaces.

Multilevel Page Tables

- Avoid keeping all the page tables in memory all the time.
- In particular, those that are not needed should not be kept around.

Inverted Page Tables

- Instead of making a very large page table, we can instead use an inverted page table with one entry per page frame. E.g. PowerPC, the UltraSPARC, and the Itanium.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

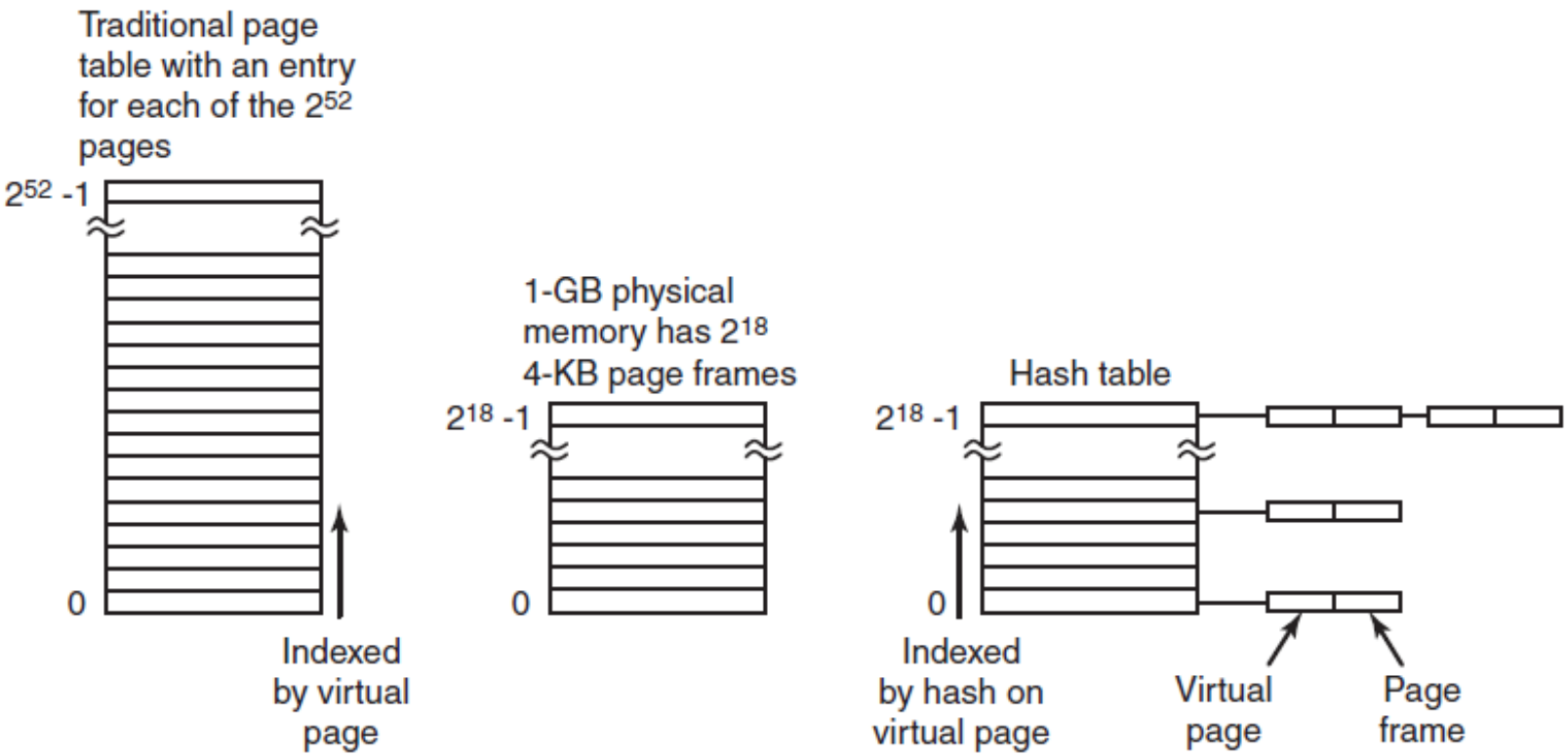
Page Tables for Large Memories
Inverted Page Tables

- **Page table entry contains**
  - Process ID (table shared between all processes)
  - Virtual page number
- **Table indexed by physical page number (indexed by hash on virtual page)**
- **All the virtual pages currently in memory that have the same hash value are chained together**

For example, with 64-bit virtual addresses, a 4-KB page size, and 4 GB of RAM, an inverted page table requires only 1,048,576 entries 4GB/4KB = 2^32B/2^12B = 2^20. The entry keeps track of which (process, virtual page) is located in the page frame.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Page Tables for Large Memories
Inverted Page Tables



Comparison of a traditional page table with an inverted page table.

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Page Tables for Large Memories
Inverted Page Tables

In virtual memory systems, it’s possible for two processes to use the same physical page frame even if their code is different.

This can happen due to shared libraries, memory optimization strategies, or non-executable data sharing. The key is that the virtual address spaces of the processes map to the same physical frames, but the content and usage of these frames can differ.

Page Tables for Large Memories
Inverted Page Tables

P	PID
5	B

P	Offset

PF	PID	P	Present/Absent bit
0			0
1			0
2	A	3	1
3			0
4			0
5	A	18	1
6			0
7			0
8	B	5	1
9			0
10			0
11			0
12			0
13			0
14			0
15	B	2	1

PF	Offset

8	Offset

Page Tables for Large Memories
Inverted Page Tables

Summary

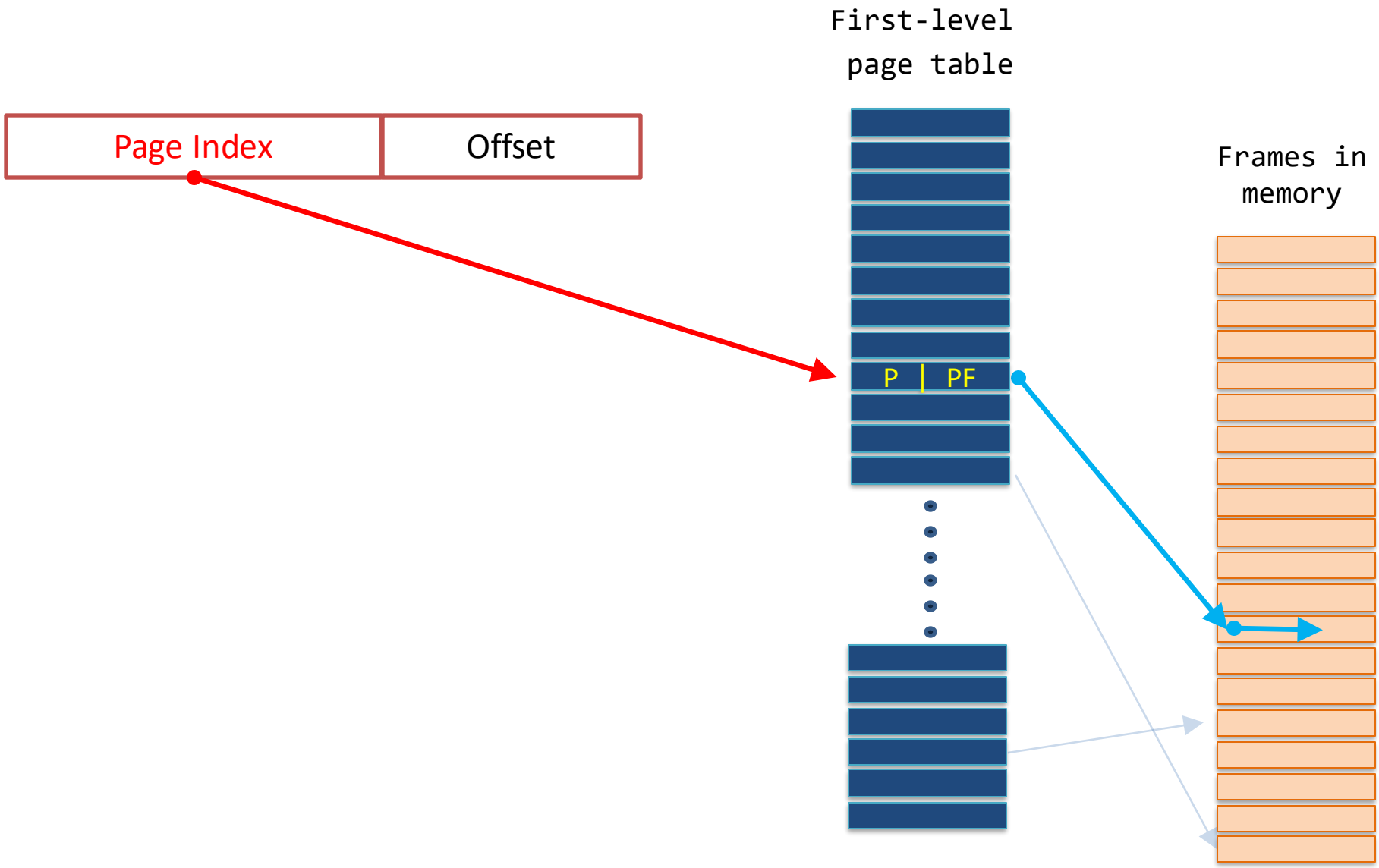
Aspect	Page Table	Inverted Page Table
Memory Overhead	Higher, as each process has its own page table	Lower, as there is only one global table
Lookup Efficiency	Direct mapping, usually faster	Requires searching, which can be slower
Isolation	Good isolation between processes	Shared table, requires process ID for lookups
Complexity	More complex due to multiple tables	Less complex in terms of table management
Scalability	Can become large with many processes	Potentially better for systems with many processes but can be slower with large tables

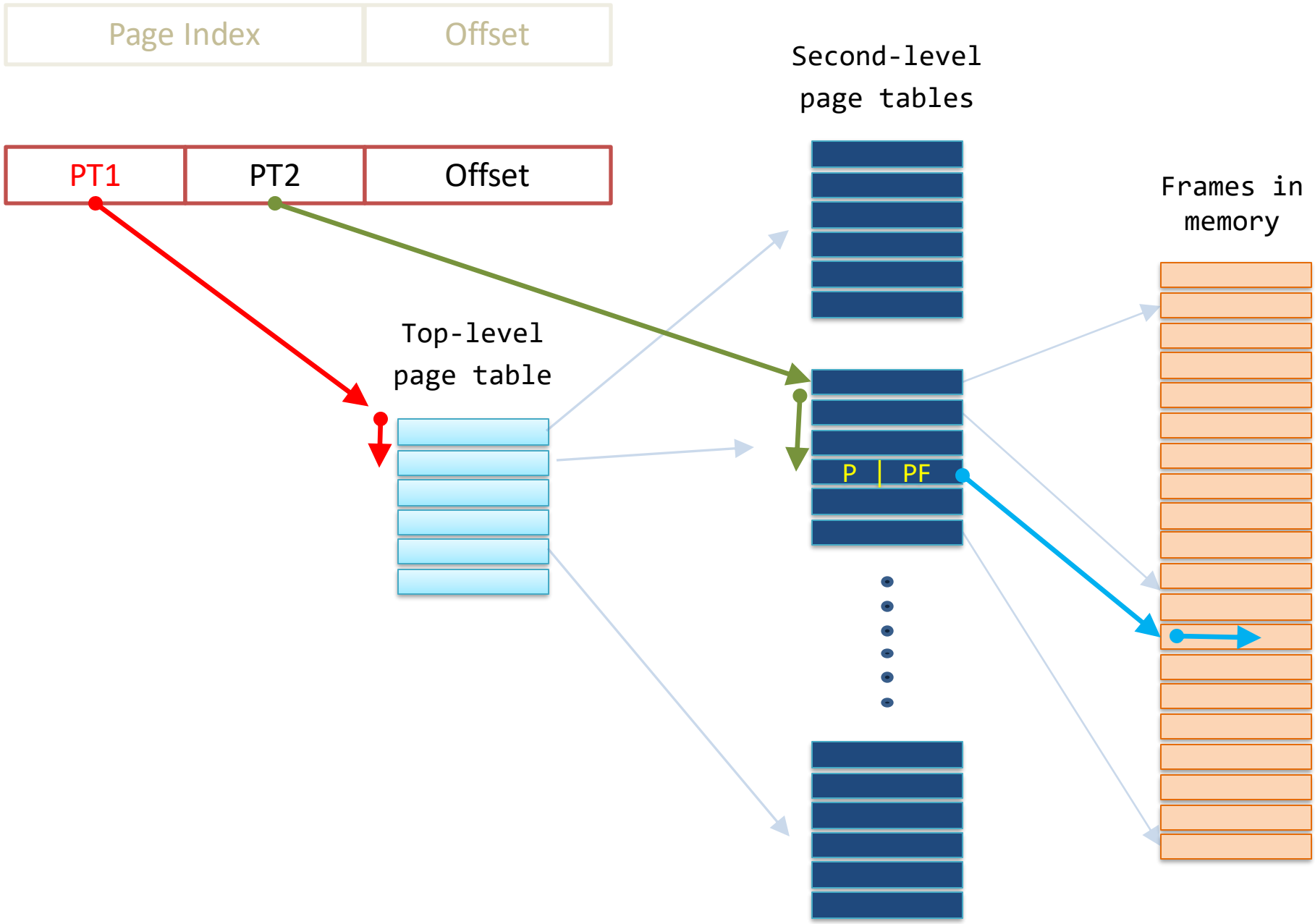
Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Page Tables for Large Memories

Multilevel Page Tables

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.


First-level
page table
Page Index
Offset
P
PF
Frames in
memory


Page Index
Offset
PT1
PT2
Offset
Second-level page tables
Frames in memory
Top-level page table
P
PF

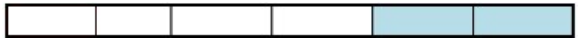
Page Tables for Large Memories

Multilevel Page Tables

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19
5	20	21	22	23
6	24	25	26	27
7	28	29	30	31
8	32	33	34	35
9	36	37	38	39
10	40	41	42	43
11	44	45	46	47
12	48	49	50	51
13	52	53	54	55
14	56	57	58	59
15	60	61	62	63

Page size = 4 = 2^2
The number of pages = 16 = 2^4
Total virtual address space = 16x4 = 64 = 2^6

Single Level Page Table

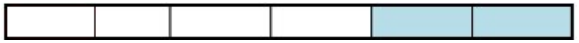


Page Tables for Large Memories

Multilevel Page Tables

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11
3	12	13	14	15
4	16	17	18	19
5	20	21	22	23
6	24	25	26	27
7	28	29	30	31
8	32	33	34	35
9	36	37	38	39
10	40	41	42	43
11	44	45	46	47
12	48	49	50	51
13	52	53	54	55
14	56	57	58	59
15	60	61	62	63

Single Level Page Table

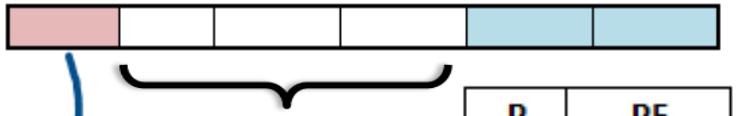


P	PF
0	3
1	6
2	8
3	9
4	12
5	10
6	x
7	x
8	x
9	x
10	x
11	x
12	x
13	x
14	x
15	x

Page Tables for Large Memories

Multilevel Page Tables

Two Level Page Table



P	PF
0	3
1	6
2	8
3	9
4	12
5	10
6	x
7	x
8	x
9	x
10	x
11	x
12	x
13	x
14	x
15	x

0

1

0
1

P	PF
0	3
1	6
2	8
3	9
4	12
5	10
6	x
7	x
0	x
1	x
2	x
3	x
4	x
5	x
6	x
7	x

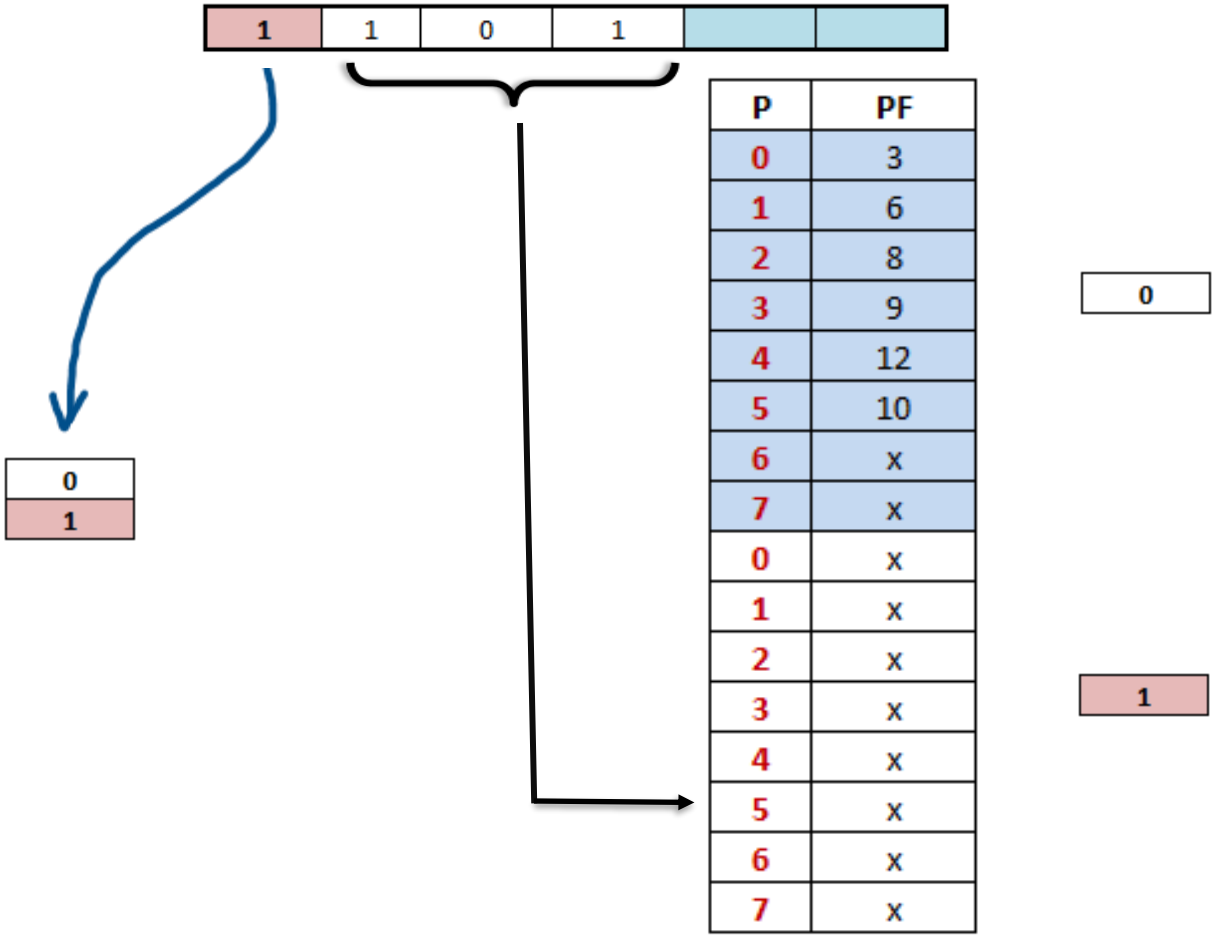
0

1

Page Tables for Large Memories

Multilevel Page Tables

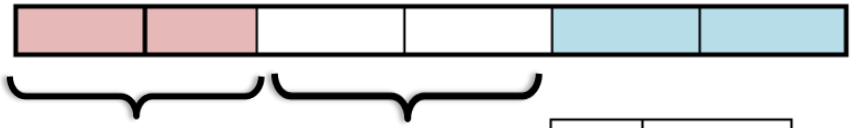
Two Level Page Table



Page Tables for Large Memories

Multilevel Page Tables

Two Level Page Table



P	PF
0	3
1	6
2	8
3	9
4	12
5	10
6	x
7	x
8	x
9	x
10	x
11	x
12	x
13	x
14	x
15	x

0

1

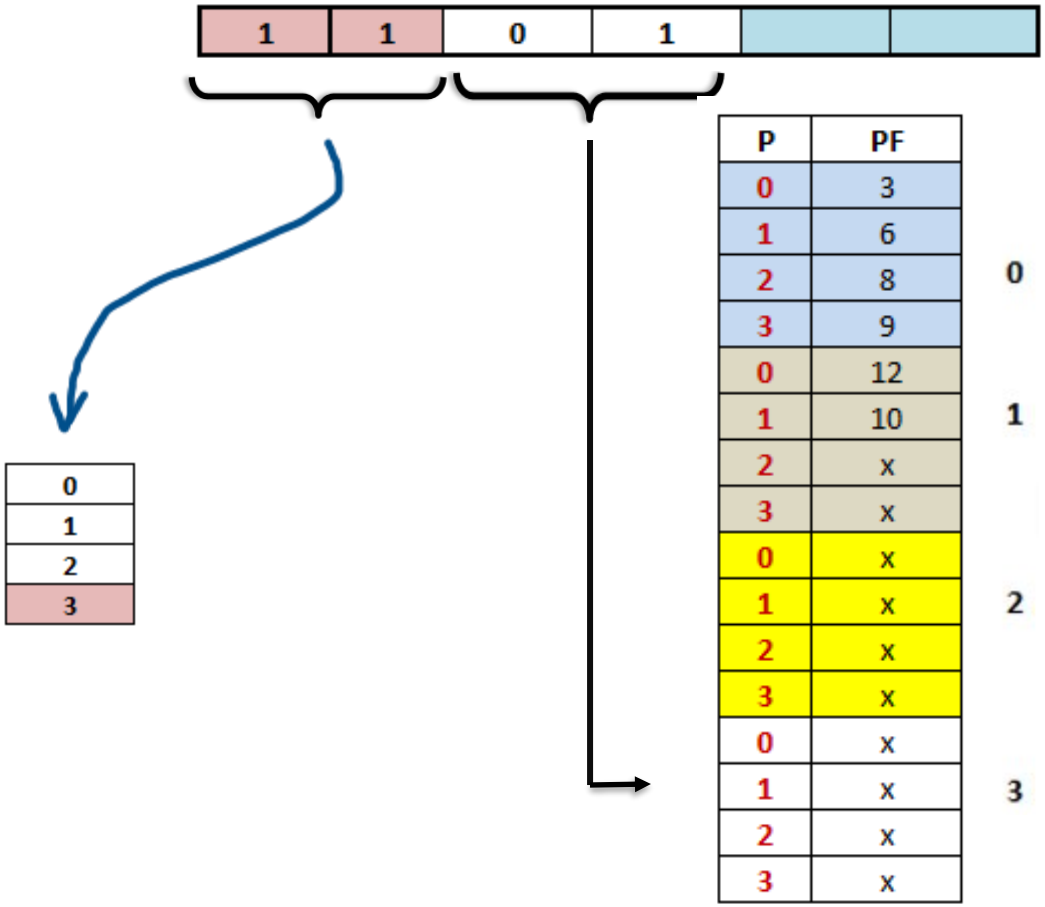
2

3

0
1
2
3

P	PF	
0	3	
1	6	
2	8	0
3	9	
0	12	
1	10	1
2	x	
3	x	
0	x	
1	x	2
2	x	
3	x	
0	x	
1	x	3
2	x	
3	x	

Page Tables for Large Memories

Multilevel Page Tables

Two Level Page Table